



A versatile FEM framework with native GPU scalability via globally-applied AD

Mohit Pundir¹*, Flavio Lorez¹, David S. Kammer¹

Institute for Building Materials, ETH Zurich, Switzerland

ARTICLE INFO

Keywords:

Finite-element analysis
Automatic differentiation
High-performance computing
Graph coloring
GPUs

ABSTRACT

Energy-based finite-element formulations provide a unified framework for describing complex physical systems in computational mechanics. In these energy-based methods, the governing equations can be obtained directly by considering the derivatives of a single global energy functional. While Automatic Differentiation (AD) can be used to automate the generation of these derivatives, current frameworks face a clear trade-off based primarily on the scale upon which the AD method is applied. Globally applied AD offers high expressivity but cannot currently be scaled to large problems. Locally applied AD scales well through traditional assembly methods, but the variety of physics and couplings that the framework can easily represent is more limited than the global approach. Here, we introduce an energy-centric framework (*tatva*) that defines the physics of a problem as a single global functional and applies AD globally to generate residual and tangent operators. By leveraging Jacobian-vector products for matrix-free solvers and coloring-based sparse differentiation for materializing sparse tangent stiffness matrices when needed, our flexible design scales linearly with the problem size on GPUs. We demonstrate that our framework can handle large problems (with millions of degrees of freedom) without memory exhaustion. Additionally, it offers a unified, fully differentiable methodology that can address a wide range of problems, including multi-point constraints, mixed-dimensional coupling, and the incorporation of neural networks, while maintaining high performance and scalability on modern GPU architectures.

1. Introduction

Many variational finite-element methods used in computational mechanics are built on an energy-based formulation [1–3]. Based on the principle of minimum potential energy, the governing equations for the equilibrium state are derived as the first- and second-order derivatives of a single scalar energy functional. This approach provides a compact and physically consistent description of complex systems. Representing a problem via an energy functional enables the seamless integration of nonlinear material laws, multiphysics couplings, and constraints within a unified finite-element framework.

The computation of the required derivatives of the energy functional is often non-trivial for complex problems. Many prominent finite-element packages rely on symbolic differentiation (SD) to automate this process [4,5]. While highly effective for many standard mechanics problems, SD fundamentally requires closed-form expressions and is ill-suited to the differentiation of algorithmic code containing conditional logic or iterative loops. Automatic differentiation (AD) [5,6] offers a systematic way to compute derivatives for such complex scenarios. By algorithmically applying the chain rule directly to the executing code, AD automatically computes derivatives up to machine precision, seamlessly handling conditionals and loops [7,8]. Consequently, by delegating the evaluation

* Corresponding author.

E-mail addresses: mpundir@ethz.ch (M. Pundir), florez@ethz.ch (F. Lorez), dkammer@ethz.ch (D.S. Kammer).

<https://doi.org/10.1016/j.cma.2026.119118>

Received 12 February 2026; Received in revised form 24 April 2026; Accepted 22 May 2026

Available online 1 June 2026

0045-7825/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

of derivatives to the computational framework, AD reduces implementation effort and enables rapid exploration of novel physical models. This has led to the widespread adoption of AD in scientific computing [9–15]. In principle, AD allows finite-element formulations to be expressed directly at the level of the energy functional.

In practice, AD has been applied in two distinct ways in finite-element methods. One approach applies AD to a global discrete energy functional [16]. In this setting, the residual vector and tangent operator are obtained by differentiating a single scalar-valued energy with respect to all degrees of freedom, see Fig. 1. This approach provides a direct correspondence between the variational formulation and its discrete realization. It allows for the natural expression of global constraints, nonlocal interactions, mixed-dimensional couplings, and unconventional discretizations. However, direct differentiation of a global energy produces dense derivative operators leading to a memory footprint and computational costs that scale quadratically with the number of degrees of freedom, *i.e.* $\mathcal{O}(N^2)$. This overhead typically limits the applicability of the global approach to small-scale problems or specific research applications.

A second approach restricts AD to local quantities, typically at the element or quadrature-point level [12,17]. Here, AD is used to compute local residuals and tangent contributions, which are then assembled into global sparse operators using standard finite-element procedures, see Fig. 1. This approach addresses memory limitations by preserving the sparsity of the tangent stiffness matrix, thereby enabling scalability to large problem sizes. Consequently, the application of AD to local quantities has become the prevailing approach in practical finite-element implementations that rely on AD [18].

However, restricting differentiation to local quantities imposes significant limitations on the simplicity and versatility of the method. By shifting AD to the element level, the global problem structure is no longer directly represented in the differentiation process. Instead, the global system must be recovered through element-wise assembly. As a result, this approach inherits the element-centric abstractions of classical finite-element libraries [4,19–22]. This makes it difficult to use this framework to analyze systems involving local and non-local constraints (*e.g.*, contact or multi-point constraints), mixed-dimensional couplings (*e.g.*, cohesive-fracture or beam-solid interaction) or multiphysics systems with disparate degrees of freedom (*e.g.*, thermo-mechanical coupling). While AD simplifies the math within an element, it does not fundamentally change how the global problem is formulated or executed.

The element-level approach also retains the traditional compute-assemble-solve workflow, see Fig. 1. Local contributions are accumulated into global sparse operators via “scatter-add” operations that rely on unstructured memory access patterns. While effective on CPUs, this paradigm is not well suited to modern accelerator hardware such as GPUs or TPUs. The irregular memory access required by scatter-add assembly conflicts with the stream-oriented execution model of GPUs. This leads to memory-bandwidth bottlenecks that limit performance and hence scalability. As a result, current research often focuses on optimizing the assembly process [20,23]; such approaches overlook more substantial improvements that could be obtained by reconsidering the representation of the global variational problem itself.

In summary, existing approaches illustrate a clear trade-off. Global energy-based differentiation offers high expressive power and a transparent connection to the variational formulation, but lacks the computational practicality needed for large-scale simulations. Conversely, element-level differentiation achieves scalability and performance but constrains the class of physical problems that can be expressed naturally. This leaves an unresolved challenge in AD-based finite-element frameworks: achieving both high-level expressiveness to tackle diverse problems and computational efficiency within a single framework.

Here, we revisit global energy-centric formulations in the context of modern AD and high-performance computing tools. We aim to retain the flexibility of global formulations while enabling the efficient evaluation of residuals and tangent operators for large-scale problems. To achieve this, our framework utilizes graph-coloring-based sparse differentiation and Jacobian-vector products (JVPs) to bypass the memory bottlenecks of dense differentiation. We present this approach through `tatva` [24], a monolithic, energy-centric library. We demonstrate that this framework is both broadly applicable to diverse physical problems and highly scalable on GPU architectures.

2. Proposed framework

We propose a framework that adopts an energy-centric philosophy, in which the behavior of a physical system is defined by a scalar potential energy. Following a variational approach [1], residuals and tangent operators are obtained as the first and second derivatives of the total energy Ψ ,

$$\mathbf{r}(\mathbf{u}) := \nabla \Psi(\mathbf{u}), \quad \mathbf{K}(\mathbf{u}) := \nabla^2 \Psi(\mathbf{u}). \quad (1)$$

where $\mathbf{u}$ denotes a physical field such as displacement, temperature, or velocity. While this mathematical foundation is standard in variational finite-element formulations (see *e.g.* [16]), the proposed framework adopts a different computational architecture. In contrast to existing AD-based approaches [12,17,18], which embed differentiation within element-level assembly loops, a monolithic structure is introduced here that decouples the description of the physical problem from the numerical backend.

To realize the monolithic architecture, we discretize the continuous functional $\Psi(\mathbf{u})$ as a sum over the finite-element mesh. We evaluate the global potential by aggregating local element contributions computed through numerical integration at quadrature points,

$$\Psi_h(\mathbf{u}_h) = \sum_{e=1}^{N_{\text{el}}} \sum_{q=1}^{N_q} \psi(\nabla \mathbf{u}_h^e(\xi_q)) w_q \det(\mathbf{J}^e(\xi_q)) \quad (2)$$

where ψ is the local energy density, w_q and ξ_q denote the quadrature weights and positions, and $\mathbf{J}^e$ is the Jacobian of the mapping from the reference to the physical element.

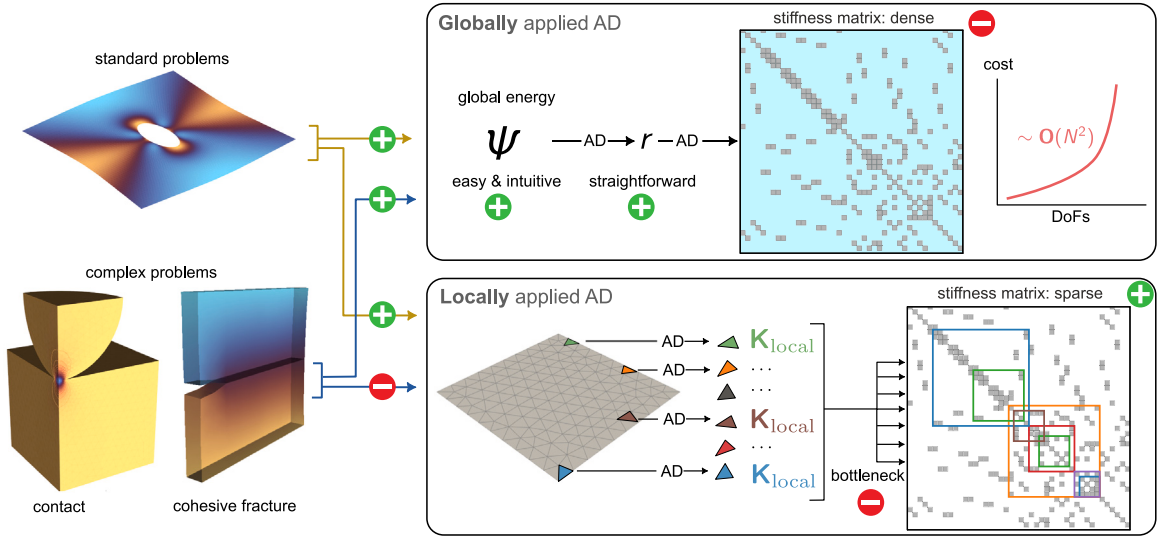


Fig. 1. Schematic of global and local approaches in AD-based finite-element frameworks. The global approach differentiates the total energy to obtain residual and tangent operators, enabling complex problems such as contact between deformable bodies and interface fracture, but with quadratic memory and computational costs due to the construction of a dense tangent stiffness matrix. The local approach applies AD at the element level to obtain local tangent operators, then uses standard scatter-add assembly to form global operators, reducing memory cost but making complex problems harder to model and less expressive.

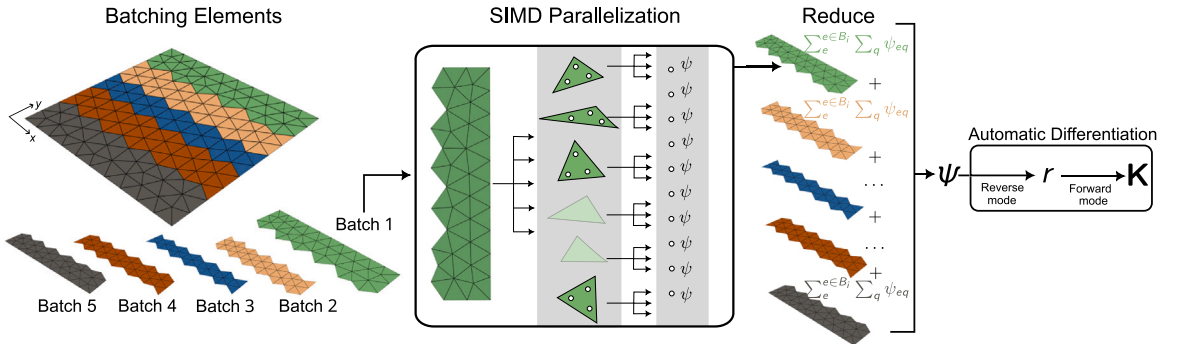


Fig. 2. Schematic of the batched monolithic engine. To mitigate memory bottlenecks on GPUs, the global mesh is partitioned into contiguous element batches. Each batch is processed via SIMD-parallelized kernels that evaluate the local energy density ψ simultaneously at each quadrature point from all elements in that batch and reduce them to a potential energy for that batch. Energy contribution from each batch is further reduced to get the global potential energy Ψ_h . The framework then recovers the global residual vector r (via reverse-mode AD) and the stiffness operator K (via forward-mode AD) directly from this scalar graph, bypassing the legacy element-level matrix assembly entirely.

In the proposed framework, we handle the transition from the mathematical formulation to hardware execution through batched computation. Instead of iterating over elements individually, we group elements into contiguous batches and process them collectively, see Fig. 2. By unrolling the element loop into smaller groups, as shown in Algorithm 1, we ensure the peak memory footprint depends on the batch size rather than the total mesh size, preventing the materialization of the full computational graph for large problems. This execution strategy aligns naturally with the SIMD (Single Instruction, Multiple Data) model of modern processors and shifts the finite-element computation toward a compute-bound regime. As a result, the framework maintains high hardware occupancy through internal vectorization while exposing a high degree of parallelism.¹ As shown in Section 5, this formulation sustains constant throughput and linear $O(N)$ scaling on GPUs, even for simulations with millions of degrees of freedom.

We apply AD directly to the global scalar potential $\Psi_h(u_h)$ to derive the residual vector and the tangent stiffness matrix. By differentiating the scalar energy functional, we ensure that all computational operators remain consistent with the fundamental

¹ Other recent frameworks also use element batching [25]. However, they use batching to speed up traditional scatter-add assembly on CPU and explicitly abandon AD. In contrast, we use batching solely to construct a single global energy functional. This allows us to apply global AD and completely bypass scatter-add memory conflicts.

Algorithm 1 Batched Energy Computation**Require:** Global vector $\mathbf{u}_h \in \mathbb{R}^{N_{\text{dofs}}}$, geometric dimension d , degrees of freedom per node m .**Ensure:** Total potential $\Psi_h \in \mathbb{R}$.

- 1: Let $a \in \{1, \dots, n_{\text{en}}\}$ denote the local node index per element.
- 2: Let $q \in \{1, \dots, N_q\}$ denote the local quadrature point index per element.
- 3: Reshape to nodal representation: $\mathbf{u}_h \rightarrow \mathbf{u} \in \mathbb{R}^{N_{\text{nodes}} \times m}$.
- 4: **Partition** elements $\{1, \dots, N_{\text{el}}\}$ into batches $\{B^k\}$ of size N_B .
- 5: $\Psi_h \leftarrow 0$
- 6: **for each** batch B^k **do** ▷ e.g., via `jax.lax.map`
- 7: **Gather** geometry factors $\det(\mathbf{J}^k(\xi_q))$. ▷ Shape: $[N_B, N_q]$
- 8: **Gather** element nodal values $\mathbf{u}^k$. ▷ Shape: $[N_B, n_{\text{en}}, m]$
- 9: **Gather** element basis gradients $\nabla \mathcal{N}^k(\xi_q)$ ▷ Shape: $[N_B, N_q, n_{\text{en}}, d]$
- 10: **Compute** field gradients at quadrature points:

$$\nabla \mathbf{u}^k(\xi_q) = \sum_{a=1}^{n_{\text{en}}} \mathbf{u}_a^k \otimes \nabla \mathcal{N}_a^k(\xi_q).$$
▷ Shape: $[N_B, N_q, m, d]$
- 11: **Evaluate** strain energy density $\psi^k(\xi_q) = \psi(\nabla \mathbf{u}^k(\xi_q))$. ▷ Shape: $[N_B, N_q]$
- 12: **Integrate** element energies:

$$\Psi^k = \sum_{q=1}^{N_q} \psi^k(\xi_q) w_q \det(\mathbf{J}^k(\xi_q)).$$
▷ Shape: $[N_B]$
- 13: **Accumulate** $\Psi_h \leftarrow \Psi_h + \sum \Psi^k$. ▷ Scalar

variational formulation, see Eq. (1). Operating at the level of the global energy allows us to express the full physics in a single computational graph, independent of any element-wise assembly logic. To make this approach computationally efficient, we select the AD mode based on the mapping dimensionality at each stage of the pipeline, thereby balancing computational cost and memory usage.

To compute the residual vector $\mathbf{r}$, we use reverse-mode AD. The global energy functional maps a high-dimensional input field $\mathbf{u}$ to a single scalar value, making reverse-mode AD the most efficient choice for this $N_{\text{dofs}} \rightarrow 1$ mapping [26]. Using this mode, we obtain the full residual vector at a computational cost that is only a small multiple of the cost of a single energy evaluation, independent of the total number of degrees of freedom.

To compute the tangent stiffness matrix $\mathbf{K}$, we need to evaluate second-order derivatives of the global energy functional. A direct application of AD at this level typically produces a dense Hessian, whose storage scales quadratically with the number of degrees of freedom. For large-scale problems, this dense representation quickly becomes infeasible due to high memory consumption. Consequently, a naive global differentiation of the energy functional does not provide a practical path towards scalable stiffness evaluation.

To address this challenge, we use two complementary strategies that operate directly on the global energy functional. When only the action of the stiffness matrix is required, for example, in iterative solvers or time integration schemes, we rely on matrix-free Jacobian-vector products, as described in Section 2.1. In contrast, when an explicit stiffness matrix is needed, such as for eigenvalue analyses or saddle-point problems, we recover a sparse representation of the tangent operator using graph-coloring-based differentiation, as discussed in Section 2.2. Together, these strategies allow the framework to remain scalable while supporting a broad range of problem classes.

2.1. Matrix-free Jacobian-Vector Products (JVP)

For matrix-free solvers, we compute the action of the tangent stiffness matrix expressed as Jacobian-Vector Products (JVP), without explicitly forming the matrix. We obtain JVPs by applying forward-mode AD to the residual function,

$$\underbrace{\delta \mathbf{r}}_{\mathbf{K}} = \frac{\partial \mathbf{r}}{\partial \mathbf{u}} \delta \mathbf{u} \Rightarrow \delta \mathbf{r} = \text{JVP}(\delta \mathbf{u}). \quad (3)$$

This operation evaluates the directional derivative of the residual in the direction $\delta \mathbf{u}$ and naturally from the monolithic formulation, in which differentiation is applied at the level of the global functional.

Matrix-free JVPs are well-suited for large-scale simulations in which iterative solvers or time integration schemes require repeated applications of the stiffness operator but do not require explicit access to the stiffness matrix. By avoiding the materialization of $\mathbf{K}$, this approach substantially reduces memory usage, and the cost of a single JVP evaluation remains comparable to that of computing the residual. Consequently, JVP evaluations scale linearly with the number of degrees of freedom.

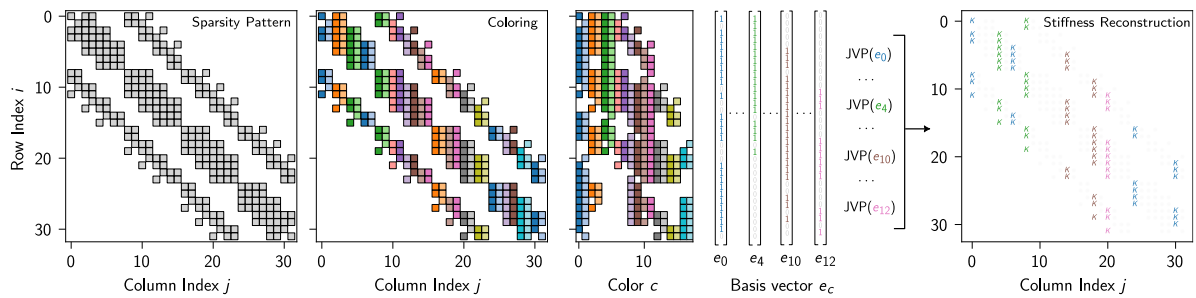


Fig. 3. Schematic view of sparse differentiation via coloring algorithm. (from left-to-right) Foremost, we construct the sparsity pattern of a tangent stiffness matrix using mesh connectivity. The gray boxes represent the non-zero entries, and white represents zero entries. Next, we color the non-zero entries, i.e. the gray boxes, using the distance-2 greedy algorithm. Non-zero entries in each color group are independent of each other and do not share any degrees of freedom. We group the entries color-wise into a compressed form with size $N_{\text{dofs}} \times c$. Each column represents a basis vector (or composite perturbation vector) with 1 at non-zero entries and 0 in all other rows. Finally, applying a single Jacobian-vector product to each basis vector allows us to construct a compressed Jacobian matrix $\mathbf{J}_{\text{comp}}$ of size $N_{\text{dofs}} \times c$ which is then used to reconstruct the sparse tangent stiffness matrix $\mathbf{K}$. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

2.2. Sparse differentiation using coloring

For problems or solvers that rely on an explicit representation of the tangent stiffness matrix, we construct one using sparse differentiation [27,28] with *matrix coloring* [29]. In principle, this can be done by evaluating the JVPs of the residual $\mathbf{r}$ with respect to a set of basis vectors. Evaluating a JVP with a canonical basis vector $\mathbf{e}_i$ (1 at i th column, 0 elsewhere) yields the i th column of $\mathbf{K}$. Repeating this procedure for all n degrees of freedom would recover the full matrix, but at a prohibitive cost of $n \times t_{\text{JVP}}$.

We reduce this cost by exploiting the sparsity structure of $\mathbf{K}$, as illustrated in Fig. 3 and outlined in Algorithm 2. Using distance-based graph coloring of the finite-element connectivity, we partition the degrees of freedom into c independent sets such that no two degrees of freedom within the same set contribute to the same row of the stiffness matrix. Each color, therefore, defines a composite perturbation vector, and a single JVP evaluated with this vector recovers multiple non-overlapping columns of $\mathbf{K}$ simultaneously. The total cost of reconstructing the stiffness matrix is thus proportional to the number of colors, c , rather than the total number of degrees of freedom. For standard finite-element discretizations, c is bounded by a constant that depends only on the element type and local connectivity and is independent of the global mesh size (see Appendix A for more details). In practice, this results in a small number of JVP evaluations, even for meshes with millions of degrees of freedom. Because all degrees of freedom within a color are independent, each color pass can be evaluated in parallel without memory conflicts, allowing the compiler to exploit hardware parallelism efficiently. Furthermore, each pass to JVP can be evaluated in parallel because there is no memory conflict among the groups.

Algorithm 2 Sparse differentiation using coloring

Require: Evaluation point $\mathbf{u}$, residual function $\mathbf{r}$, DOF color mapping $\mathbf{c}$, number of colors n_{colors} .

Ensure: Compressed Jacobian $\mathbf{J}_{\text{comp}}$.

- 1: **Initialize** color IDs: $\{0, 1, \dots, n_{\text{colors}} - 1\}$.
- 2: **for each** $c \in$ color IDs **do** ▷ Implemented via `jax.lax.scan`
- 3: **Generate** seed vector $\mathbf{e}$: $e_j = 1$ if $c_j = c$, else 0.
- 4: **Compute** JVP: $\mathbf{y} = \nabla \mathbf{r}(\mathbf{u}) \cdot \mathbf{e}$. ▷ Directional derivative
- 5: **Store** $\mathbf{y}$ as column c of $\mathbf{J}_{\text{comp}}$.
- 6: **return** $\mathbf{J}_{\text{comp}}$ with shape $[N_{\text{dofs}}, n_{\text{colors}}]$.

Require: Compressed Jacobian $\mathbf{J}_{\text{comp}}$, row pointers, column indices, color mapping $\mathbf{c}$.

Ensure: Sparse stiffness matrix $\mathbf{K}$ in sparse format.

- 1: **Expand** row pointers to generate row indices i for every non-zero entry from sparsity pattern.
 - 2: **Retrieve** column indices j from sparsity pattern.
 - 3: **Map** column indices to colors: $c = \mathbf{c}[j]$.
 - 4: **Extract** values: $K_{ij} = \mathbf{J}_{\text{comp}}[i, c]$.
 - 5: **Construct** sparse matrix $\mathbf{K}$ from indices (i, j) and values K_{ij} .
 - 6: **return** $\mathbf{K}$.
-

The proposed framework, as described in this section, defines a monolithic, energy-centric approach in which physical models are expressed entirely in terms of a global scalar potential and differentiated systematically using automatic differentiation. By

combining batched evaluation of the energy functional with global differentiation strategies, the framework avoids traditional element-wise assembly while remaining compatible with modern hardware architectures. The availability of both matrix-free Jacobian-vector products and sparse stiffness recovery provides flexibility in how second-order derivatives are accessed, allowing the same formulation to support a wide range of problem classes without modifying the underlying physics description.

3. Versatility of the proposed framework

The proposed framework is designed to isolate the stiffness construction from the specific physical model, allowing a wide range of problems to be expressed within a single computational architecture. This design choice enables the same formulation to be applied across diverse classes of problems, as demonstrated in the examples presented in Section 6. Although the framework is formulated in terms of a potential energy functional, it is not restricted to conservative systems. As discussed in Section 3.1, the architecture naturally extends to non-variational problems, such as fluid transport or non-associative plasticity. Furthermore, the same monolithic structure facilitates the integration of data-driven models without modifying the core formulation, as described in Section 3.2.

3.1. Generalization to non-variational systems

Our architecture handles the non-variational problems by extending the energy-centric approach to the principle of virtual work, as we show through an example in Section 6.5. We introduce a scalar virtual work functional $\mathcal{W}(\mathbf{u}, \mathbf{v})$, which represents the work done by internal and external forces against a test function $\mathbf{v}$:

$$\mathcal{W}(\mathbf{u}, \mathbf{v}) = \int_{\Omega} \boldsymbol{\sigma}(\mathbf{u}) : \nabla \mathbf{v} d\Omega - \int_{\Omega} \mathbf{f} \cdot \mathbf{v} d\Omega \quad (4)$$

The discretized form follows the same logic as the energy formulation by summing scalar contributions over quadrature points:

$$\mathcal{W}_h(\mathbf{u}_h, \mathbf{v}_h) = \sum_{e=1}^{N_{el}} \sum_{q=1}^{N_q} (\boldsymbol{\sigma}(\mathbf{u}_h^e(\xi_q))) : \nabla \mathbf{v}_h^e(\xi_q) w_q \det(\mathbf{J}^e(\xi_q)) \quad (5)$$

Our batched computation algorithm works exactly the same way for these problems. The framework simply aggregates the scalar product $\boldsymbol{\sigma} : \nabla \mathbf{v}$ instead of the energy density Ψ_h . We then recover the global residual and tangent stiffness by differentiating this functional with respect to the test and trial functions:

$$\mathbf{r}(\mathbf{u}) := \nabla_{\mathbf{v}} \mathcal{W}(\mathbf{u}, \mathbf{v}) \Big|_{\mathbf{v}=0}, \quad \mathbf{K}(\mathbf{u}) := \nabla_{\mathbf{u}} \mathbf{r}(\mathbf{u}) = \nabla_{\mathbf{u}}^2 \mathcal{W}(\mathbf{u}, \mathbf{v}) \Big|_{\mathbf{v}=0} \quad (6)$$

This approach ensures that the monolithic architecture supports operator fusion and GPU acceleration for non-symmetric or path-dependent physics.

3.2. Future-proofing with implicit AI coupling

The monolithic architecture also simplifies the integration of data-driven components. Traditional approaches [14,25] often treat machine learning models as external plugins, which requires batching local updates within a legacy finite-element loop. These methods rely on custom bridge code and manual derivations to communicate between the model and the solver. In contrast, our framework treats data-driven models as core physical entities, as demonstrated on two applications in Section 6.6. Whether we integrate local Neural Constitutive Models (NCM) [30–32] at the quadrature level, or domain-scale Neural Operator Element Methods (NOEM) [33], the automatic differentiation treats the neural network as an additive term in the global scalar sum. This implicit AI coupling provides a unified structure that remains scalable for high-performance computing while staying flexible enough for the next generation of AI-augmented mechanics.

4. tatva: A monolithic differentiable library

While the proposed energy-centric framework is library-agnostic, we have implemented these concepts into an open-source Python library called *tatva* [24]. Built on the JAX ecosystem [34–36], *tatva* provides a high-level API for variational mechanics while maintaining the performance of a low-level GPU-accelerated framework. The primary goal is to allow researchers to define complex physical potentials with minimal code. The library centers on the `Operator` class, which manages the relationship between mesh topology, element formulation, and the global state.

We summarize the key functionalities below:

- **Geometric Context:** The `Operator(mesh, element, batchsize)` object initializes the element connectivity, shape functions, and quadrature rules. It sets the batch size for SIMD operations such as integration and gradient calculations.

- **SIMD Batched Operations:** The `Operator` object provides high-level functions such as `op.grad` and `op.eval`, which use `jax.lax.map` to batch computations across the mesh. `op.grad` evaluates nodal gradients at all quadrature points, and `op.eval` interpolates nodal values. We use `jax.lax.map` instead of `jax.vmap` to preserve memory scalability: `vmap` materializes the full differentiable graph across all elements, quickly exhausting memory for large problems. In contrast, `jax.lax.map` iterates over element batches, bounding peak memory while still allowing the XLA compiler [37] to fuse quadrature-level operations into optimized kernels.
- **Variational Integration:** The `op.integrate` function performs the final reduction. It takes the local energy density contributions or any scalar value at all quadrature points and applies the appropriate Jacobian and quadrature weights to produce the global scalar potential Ψ_h .

The design of the library is a direct realization of the “gather-evaluate-reduce” pipeline described in Algorithm 1. A typical implementation of the total strain energy translates the pseudo-algorithm into executable code line-by-line:

```
op = Operator(mesh, element_type) # define the operator for a mesh and a element type

def total_strain_energy(u_flat):
    u = u_flat.reshape(-1, n_dofs) # reshaping
    u_grad = op.grad(u) # Compute Gradients in batches
    energy_density = strain_energy_density(u_grad) # Quadrature-level Evaluation
    return op.integrate(energy_density) # Reduction via Batched Integration

u_flat = ... # 1D array of length equal to total DoFs
total_energy = total_strain_energy(u_flat)
```

In this energy function, we pass degrees of freedom as a flattened array. This choice ensures that when the library automatically differentiates the function, it generates a residual vector and a stiffness matrix with the correct dimensions (N_{dofs} and $N_{\text{dofs}} \times N_{\text{dofs}}$) respectively.

Because `tatva` uses `jax.numpy` for its data structures, handling and slicing data is as simple as using standard NumPy. This compatibility allows researchers to express complex mathematical expressions, conditional logic, or loops easily to compute energy density from quadrature-level values. This structure makes it possible to represent sophisticated physics with minimal coding effort while keeping the implementation closely aligned with the original mathematical theory.

Furthermore, the design ensures that every derivative computed by the library, whether it is the residual $\mathbf{r}$ or the stiffness matrix $\mathbf{K}$, is derived from the same monolithic computational graph. This consistency allows `tatva` to switch between colored sparse assembly and the matrix-free JVP approach without any modifications to the user's physics code. From the energy function defined above, we compute the residual function using `jax.jacrev` directly:

```
compute_residual = jax.jacrev(total_energy)
residual = compute_residual(u_flat)
```

One can then utilize this residual function to compute the action of the stiffness matrix (see Eq. (3)) via `jax.jvp`:

```
tangent = jax.jvp(compute_residual, (u_flat), (du_flat))[1]
```

To construct the sparse stiffness matrix, `tatva` provides functionalities to generate sparsity patterns and colors using a greedy distance-2 algorithm. These colors are then used to compute the sparse matrix as described in Algorithm 2:

```
from tatva import sparse # importing sparse differentiation module

# setup sparsity and coloring
sparsity_pattern = sparse.create_sparsity_pattern(mesh, n_dofs_per_node)
colors = sparse.distance2_colors(sparsity_pattern) # vector with non-zero entries colored

# materialize the sparse stiffness matrix
K_sparse = sparse.jacfdw(compute_residual, sparsity_pattern, colors) # Algorithm 2
```

This architecture provides a truly universal infrastructure for differentiable mechanics, enabling the user to pivot between robustness and scalability at the solver level without ever revisiting the physics implementation.

5. Performance analysis

In this section, we analyze the computational efficiency and scalability of the proposed framework to construct tangent operators on modern hardware. We benchmark the framework against traditional assembly-based methods to highlight the advantages of our graph coloring strategy and matrix-free JVPs. Our analysis uses two setups: a 2D elasticity problem with linear triangular elements and a 3D elasticity problem with tetrahedral elements. By varying the mesh resolution, we assess scaling behavior across a wide range of degrees of freedom (DoFs). We measure the execution time and computational throughput, measured in million DoFs per second (million DoFs/s), for two operations: matrix-free JVP evaluation and sparse assembly via graph coloring. All benchmarks were performed on an NVIDIA A100 GPU.

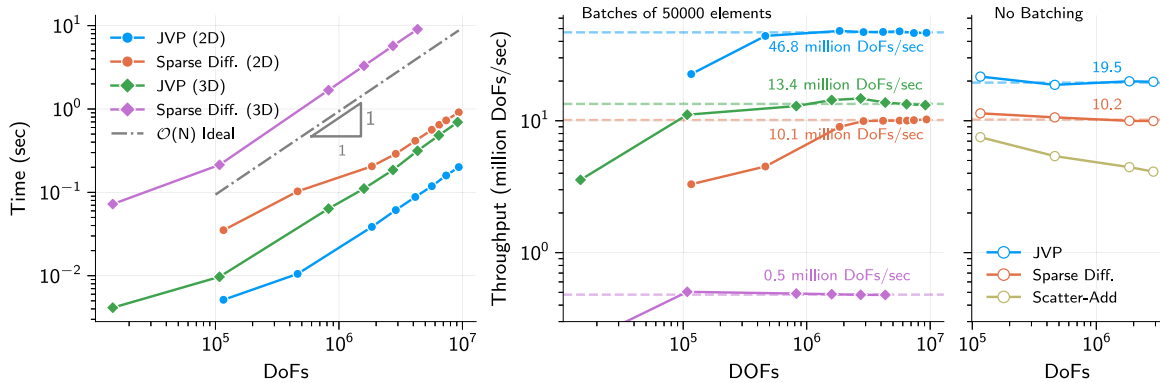


Fig. 4. Computational performance and scaling of the energy-centric framework. (left) The $\log\text{-}\log$ shows execution time for a computation with batch size of 50,000 elements. The unit slope for all methods confirms strict $\mathcal{O}(N)$ scaling. (middle) Assembly throughput (Million DoFs/sec) for the same computations, demonstrating optimal GPU utilization across three orders of magnitude. (right) Comparison to a scatter-add assembly approach similar to [12,18] for a 2D computation without batching for fair comparison. The throughput for scatter-add assembly is significantly lower, no longer constant, and constantly drops with increasing DoFs, clearly a consequence of $\mathcal{O}(N^2)$ scaling. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

5.1. Scaling and throughput analysis

We evaluate the scalability and efficiency of the proposed framework by analyzing both execution time and computational throughput with increasing DoFs up to 10 million DoFs. For both matrix-free JVP evaluation and sparse differentiation via graph coloring, the results exhibit linear $\mathcal{O}(N)$ scaling over more than three orders of magnitude in problem size (see Fig. 4-left), confirming that the proposed architecture avoids algorithmic overheads that grow with system size.

Expressed in terms of throughput (see Fig. 4-center), the results demonstrate sustained GPU efficiency across all mesh resolutions. The matrix-free formulation achieves peak throughputs of approximately 47 million DoFs/s in 2D and 13 million DoFs/s in 3D, with throughput remaining essentially constant as the problem size increases. Sparse differentiation via graph coloring exhibits lower throughput due to the multiple JVP evaluations required per color, but remains stable at approximately 10.1 million DoFs/s in 2D and 0.5 million DoFs/s in 3D.

For the 3D sparse differentiation case, the available 40 GB of GPU memory limited the maximum problem size to approximately 5.1 million DoFs. This limitation highlights the importance of memory-efficient differentiation strategies. More advanced coloring techniques [27] could further reduce the number of required colors and extend this limit.

5.2. Comparison with traditional assembly

We compare the throughput of our approach with the traditional scatter-add assembly process (see Fig. 4-right). For a fair comparison, we do not use any batching in either the reference approach or our proposed framework. The results show two significant advantages of the proposed energy-centric framework. First, both our matrix-free JVP and sparse differentiation methods achieve higher throughput than the traditional scatter-add approach. Second, unlike the constant throughput of our proposed approach, the throughput of scatter-add assembly constantly decreases with increasing number of DoFs. This performance drop is a clear indicator of the memory access bottlenecks and atomic contention inherent in element-centric assembly. In contrast, our proposed approach of evaluating the tangent operators maintains stable throughput, demonstrating that the monolithic design avoids these traditional limitations and scales effectively for large-scale engineering problems.

6. Examples

To illustrate the versatility of the proposed framework, we apply the energy-centric formulation to a diverse set of complex physical problems. These examples demonstrate that a wide range of governing equations, constraints, and couplings can be expressed within a single unified implementation without intrusive modifications to the software stack. In all cases, the formulations rely only on three basic operations, `op.grad`, `op.eval`, and `op.integrate`, introduced in Section 4, highlighting the simplicity and generality of the approach.

We begin in Section 6.1 with (linear) elasticity and essential boundary conditions to validate our framework and establish a baseline for subsequent, more complex examples. In Section 6.2, we enforce multi-point constraints using techniques such as condensation, Lagrange multipliers, and the penalty method. In Section 6.3, we demonstrate mixed-dimensional coupling, including embedded fibers and cohesive interfaces. In Section 6.5, we treat a non-variational problem formulated through the principle of virtual work. Finally, in Section 6.6, we integrate data-driven models using neural networks in the same energy-centric structure.

For readability, the code listings focus on the essential components of each formulation rather than complete scripts. Fully executable versions of all examples are available in the *tatva* documentation [38].

6.1. Handling elasticity and boundary conditions

We first consider a linear elasticity problem to illustrate how standard boundary-value problems are expressed within the proposed framework. The example consists of a plate with a circular hole at its center being subjected to a uniaxial tension, as shown in Fig. 5-left. Owing to symmetry, we model only one half of the domain, with symmetry boundary conditions imposed along the left edge. A prescribed traction is applied on the right boundary of the plate.

The problem can be formulated as the minimization of the total potential energy subject to kinematic constraints,

$$\begin{aligned} \Psi(\mathbf{u}) &= \int_{\Omega} \psi_{\epsilon}(\epsilon(\mathbf{u})) \, d\Omega - \int_{S_t} \mathbf{t} \cdot \mathbf{u} \, d\Gamma, \\ \mathbf{u}_x &= \mathbf{0} \quad \text{on } S_D, \end{aligned} \quad (7)$$

where ψ_{ϵ} denotes the strain energy density and $\mathbf{t}$ the applied traction.

In the proposed framework, this formulation translates directly into code by defining the strain energy density and assembling the total potential energy through the core operators as follows:

```
def strain_energy_density(u_grad: Array) -> Array: # linear elastic material
    eps = 0.5 * (u_grad + u_grad.T)
    sig = 2 * mu * eps + lmbda * jnp.trace(eps) * jnp.eye(2)
    return 0.5 * jnp.einsum("ij,ij->", sig, eps)

def total_energy(u_flat: Array, traction: float) -> float:
    u = reshape(u_flat)
    u_grad = op.grad(u)
    psi = strain_energy_density(u_grad)
    # op_right_edge is an Operator with the corresponding line elements
    line_integrand = op_right_edge.eval(traction * u)
    return op.integrate(psi) - op_right_edge.integrate(line_integrand)
```

Dirichlet boundary conditions are enforced by eliminating the prescribed degrees of freedom at the level of the energy functional. Instead of modifying an assembled linear system, e.g., via row/column operations or direct elimination of constrained DoFs [39], we define a reduced functional that depends only on the free degrees of freedom. The reduced displacement vector is lifted to the full field by inserting the prescribed values at constrained locations before evaluating the total energy. AD of this wrapped functional yields the consistent residual and tangent, such that the constraint handling can be carried out implicitly at the variational level:

```
free_dofs_indices = jnp.where(mesh.coords[:, 0] != 0.0)[0]

def total_energy_reduced(u_free: Array, traction: float) -> float:
    u_full = jnp.zeros(n_dofs).at[free_dofs_indices].set(u_free)
    return total_energy(u_full, traction)

residual_condensed = jax.jacrev(total_energy_reduced)
K_condensed = jax.jacfd(residual_condensed)
```

To confirm the accuracy of our model, the obtained numerical results are compared with Kirsch's analytical solution for an infinite plate with a circular hole under remote tension [40]. Excellent agreement is observed for all stress components, as shown in Fig. 5. Furthermore, we verified that the solution converges at an approximately quadratic rate under mesh refinement.

To highlight the modularity of the framework, we note that the linear elastic model could be replaced by a finite-strain hyperelastic formulation by modifying only the strain energy density ψ . All other components, including discretization, quadrature, boundary-condition handling, and solution procedure, would remain unchanged. In practice, this requires defining a new mapping from the displacement gradient to a scalar energy density,

$$\psi_{\epsilon} : \nabla \mathbf{u} \mapsto \mathbb{R},$$

which is evaluated at the quadrature points and assembled into the global energy:

```
def strain_energy_density(u_grad: Array) -> Array: # hyperelastic material
    F = jnp.eye(2) + grad_u
    C = F.T @ F # computing Cauchy strain
    ... # do stuff
    return energy_density
```

The residual and consistent tangent operators are again obtained by AD of the assembled energy functional, without requiring any further changes to the implementation.

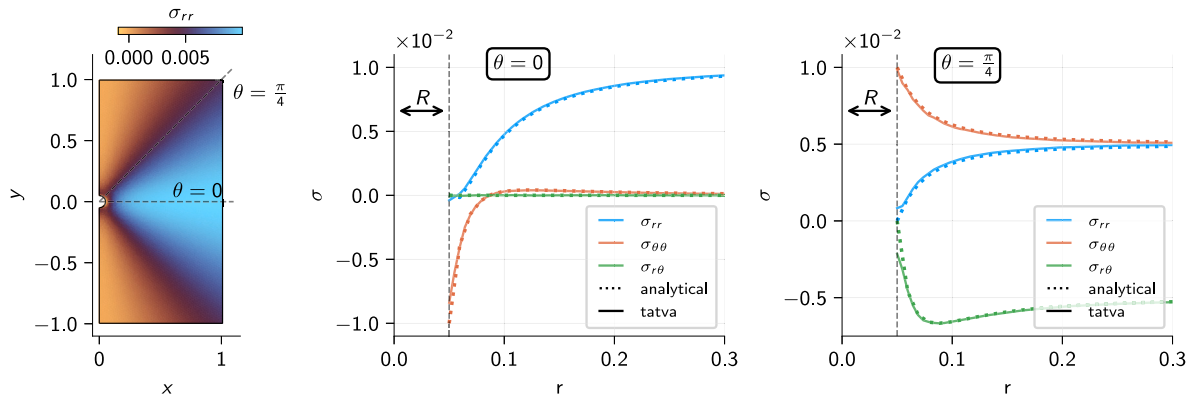


Fig. 5. Plate with a circular hole of radius $R = 0.05$ subjected to horizontal uniaxial tension $t_x = 0.01$. (left) Distribution of the radial stress component σ_{rr} in the half-domain model with symmetry boundary conditions imposed on the left edge. Dashed lines indicate radial paths corresponding to $\theta = 0^\circ$ (loading direction) and $\theta = 45^\circ$. (middle/right) Radial stress components σ_{rr} , $\sigma_{\theta\theta}$, and $\sigma_{r\theta}$ along $\theta = 0^\circ$, and $\theta = 45^\circ$, respectively, plotted as functions of the radial coordinate r . The solid lines represent the results obtained using the framework presented here; these results are compared with Kirsch's analytical solution for an infinite plate with a circular hole (dashed line).

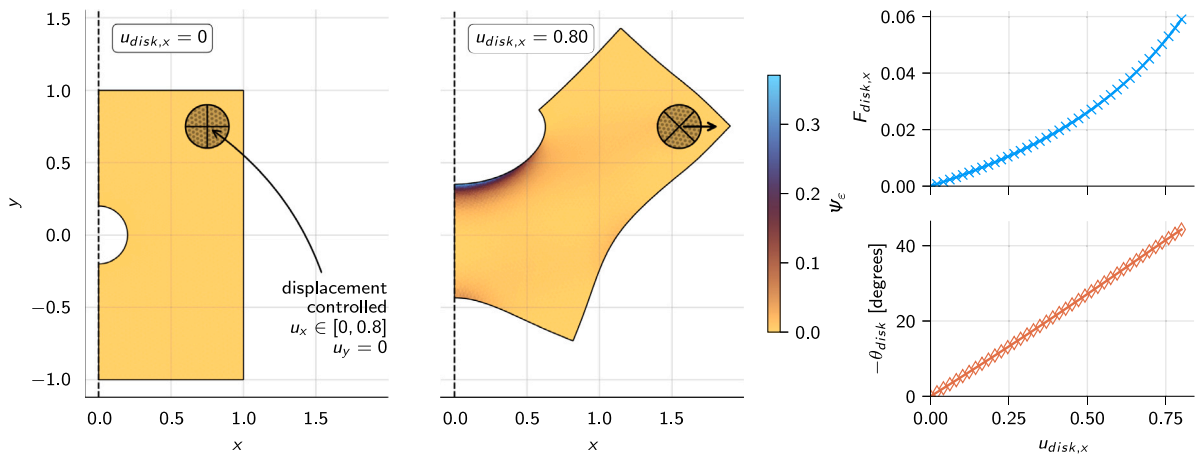


Fig. 6. Rigid-body multi-point constraint (MPC) applied to a hyperelastic plate. (left) Reference configuration with a rigid disk attached to the top-right corner. The disk motion is described by the generalized coordinates $(u_{\text{disk},x}, u_{\text{disk},y}, \theta_{\text{disk}})$, with the horizontal displacement $u_{\text{disk},x}$ prescribed and $u_{\text{disk},y} = 0$. Nodes within a radius $R_{\text{disk}} = 0.15$ of the disk center are constrained to follow rigid-body kinematics. (middle) Deformed configuration at $u_{\text{disk},x} = 0.8$, colored by the strain energy density ψ_e . A symmetry boundary condition is imposed along the left edge (dashed line). (right) Reaction force $F_{\text{disk},x}$ and rotation θ_{disk} of the rigid disk as functions of the imposed displacement $u_{\text{disk},x}$. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

6.2. Multi-point constraints

Multi-point constraints (MPCs) impose kinematic relations between the degrees of freedom associated with different nodes. Such constraints arise in a wide range of applications, including rigid-body motion, periodic boundary conditions for homogenization, and contact between deformable bodies. The proposed framework supports multiple strategies for enforcing MPCs, including static condensation, Lagrangian multiplier formulations, and penalty-based approaches, without requiring changes to the underlying differentiation pipeline.

6.2.1. Rigid body constraints using condensation

As a first illustration, we consider rigid-body motion enforced through static condensation, following the same principle used to impose essential boundary conditions in Section 6.1. In this setting, we impose linear kinematic relations on a subset of nodes so that they follow a prescribed rigid-body displacement.

Here, we again consider a plate with a hole in its center; in this case, however, there is a discontinuity going from the center of the hole upwards to the top boundary (see Fig. 6) and the material is assumed to be hyperelastic. Owing to symmetry, we only model the right half of the domain. We attach a rigid disk of radius (centered at $x = 0.75, y = 0.75$) to the top-right part of the

plate and displace it to induce deformation in the surrounding material. We describe the motion of the disk using the generalized degrees of freedom $(u_{\text{disk},x}, u_{\text{disk},y}, \theta_{\text{disk}})$, where $u_{\text{disk},x}$ and $u_{\text{disk},y}$ are prescribed, and θ_{disk} is free to adapt such that the disk does not experience any moment, and hence θ is an unknown. All nodes located within a radius $R_{\text{disk}} = 0.15$ of the disk center follow the rigid-body kinematics of the disk through MPC relations.

We construct the vector of unknowns by combining the nodal displacements and the disk degrees of freedom according to,

$$\mathbf{z} = \begin{bmatrix} \mathbf{u} \\ \mathbf{u}_{\text{disk}} \end{bmatrix}.$$

As in the case of essential boundary conditions, we enforce the constraints by defining a reduced energy functional that depends only on the free degrees of freedom. We reconstruct the full displacement field by lifting the reduced variables and applying the rigid-body kinematic relations before evaluating the energy:

```
def apply_rigid_body_bc(u_full: Array, u_disk: Array) -> Array:
    ux_c, uy_c, theta_c = u_disk # get the 3 rigid components
    cos, sin = jnp.cos(theta_c), jnp.sin(theta_c) # construct rotation matrix based on theta
    rotation_matrix = jnp.array([[cos, -sin], [sin, cos]])
    vecs = mesh.coords[disk_nodes] - disk_center
    rotated_vecs = vecs @ rotation_matrix.T # rotate vectors to the disk center
    u_rotation = rotated_vecs - vecs # get the displacement vector
    u_rigid = jnp.stack([ux_c, uy_c]) + u_rotation # add to the rigid body translation
    u_full = u_full.at[disk_nodes_x].set(u_rigid[:, 0]) # set u in x direction
    u_full = u_full.at[disk_nodes_y].set(u_rigid[:, 1]) # set u in y direction
    return u_full

def total_energy_reduced(u_free: Array, u_disk_x: float) -> float:
    u_full = jnp.zeros((n_dofs,)).at[free_dofs].set(u_free)
    u_disk = u_full[-3:] # we extract the 3 rigid body DoFs
    u_disk = u_disk.at[0:2].set([u_disk_x, 0]) # set the constraints
    u_full = apply_rigid_body_bc(u_full, u_disk)
    return total_energy(u_full)
```

Fig. 6 shows the resulting deformed configuration, together with the force–displacement and rotation–displacement responses of the rigid disk.

6.2.2. Periodic constraints for homogenization using Lagrange multipliers

This example demonstrates the implementation of periodic boundary conditions for the homogenization of a linear elastic composite using Lagrange multipliers. We consider a two-dimensional plane strain problem comprised of a skewed unit cell containing stiff circular inclusions of radius R , with elastic properties (E_r, ν_r) , embedded in a compliant linear elastic matrix with properties (E_m, ν_m) . The inclusions are arranged in a hexagonal pattern, for which homogenization theory predicts an isotropic effective macroscopic response. The example is inspired by [41].

We impose periodic boundary conditions on the displacement fluctuation field using Lagrange multipliers. The multipliers are included in the global vector of unknowns $\mathbf{z} = (\mathbf{u}, \lambda)$, and the Lagrangian functional reads

$$\mathcal{L}(\mathbf{u}, \lambda) = \Psi(\mathbf{u}) + \lambda \cdot \mathbf{g}(\mathbf{u}), \quad (8)$$

where $\Psi(\mathbf{u})$ is the total energy and $\mathbf{g}(\mathbf{u})$ collects the periodicity constraints enforcing equality of displacement fluctuations on opposing boundaries of the unit cell.

In the proposed framework, we express these constraints directly at the level of the global functional:

```
def g(u: Array) -> Array: # Returns the constraint violations
    g_left_right = u[right] - u[left] # right/left are paired boundary nodes
    g_bot_top = u[top] - u[bot] # bot/top are paired boundary nodes
    return jnp.concatenate([g_left_right, g_bot_top])

def total_energy(z_flat: Array) -> float:
    u, lambda = slice_and_reshape_z(z_flat)
    ... # computing the strain energy density
    energy_density = strain_energy_density(u_grad) # from Sec 6.1
    return op.integrate(energy_density) + jnp.dot(lambda, g(u))
```

Stationarity of $\mathcal{L}$ with respect to both $\mathbf{u}$ and λ yields a saddle-point system, which we solve monolithically using a direct linear solver. We construct the corresponding sparse tangent operator using the graph-coloring-based differentiation approach described in Section 2.2. Appendix D details the procedure used to extract the sparsity pattern of the Lagrangian functional required for sparse differentiation. Fig. 7 shows the deformed unit cell for three principal macroscopic strain states.

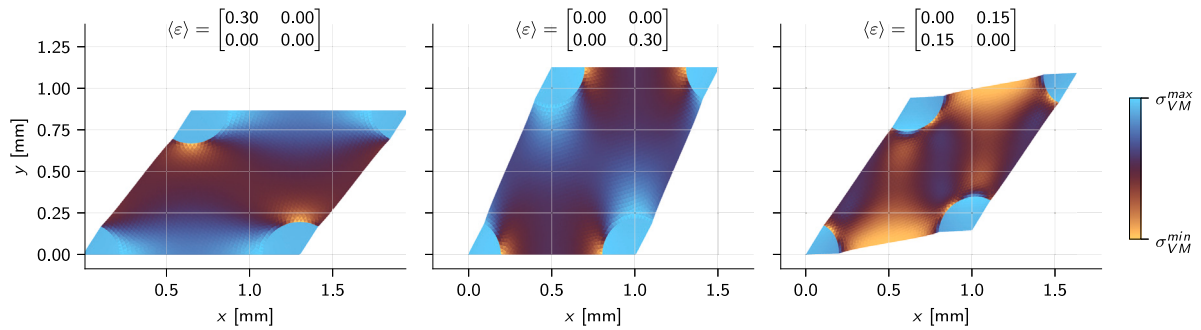


Fig. 7. Periodic homogenization of linear elasticity with a skewed unit cell. The RVE consists of two phases; a softer matrix and a stiffer inclusion. Evaluations of the microscopic response for different applied average strain tensors $\langle \epsilon \rangle$, corresponding to a pure stretch in x , y , and pure shear. This example is adopted from [41] with the same geometric and material parameters. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Beyond solving the microscopic equilibrium problem, the fully differentiable formulation enables the direct computation of the exact consistent homogenized constitutive tensor $\mathbb{C}_{\text{hom}}$. We define the macroscopic stress as the volume average of the microscopic stress obtained from the equilibrium solution corresponding to a prescribed macroscopic strain. Since the entire solve procedure is differentiable, we obtain the homogenized tangent operator by differentiating the macroscopic stress mapping with respect to the applied strain [26],

$$\sigma_{\text{hom}}(\epsilon_{\text{macro}}) \cdot$$

This approach yields the exact algorithmic homogenized tangent, including all microstructural equilibrium effects, without deriving analytical sensitivity expressions. In practice, we implement this procedure by defining a differentiable mapping from the applied macroscopic strain to the homogenized stress response:

```
def solve(eps_macro_voigt: Array) -> Array:
    eps_macro = from_voigt(eps_macro_voigt) # reshape voigt notation into 2x2 tensor
    u_tilde_flat = jnp.linalg.solve(jac(v0, eps_macro), -residual(v0, eps_macro))
    u_tilde = u_tilde_flat.reshape(-1, n_nodes)
    u = eps_hat @ x + u_tilde
    sig = jnp.mean(compute_stress(u), axis=0) # average stress response
    return to_voigt(sig) # return average stress in voigt notation
```

```
C_homogenized = jax.jacfwd(solve)(jnp.ones(3))
```

6.2.3. Contact constraints using penalty approach

We next consider a three-dimensional contact problem to illustrate how the proposed framework handles geometric constraints that introduce non-local couplings between independently discretized bodies. We model a linear elastic sphere indenting a linear elastic cube (see Fig. 8-left) and compare the numerical results with the analytical Hertz solution for frictionless contact [42].

We discretize the two bodies independently and apply essential boundary conditions using condensation, as described in Section 6.1. We fix the bottom surface of the cube in the z -direction and prescribe a vertical displacement on the top surface of the sphere octant. The interaction between the two bodies is introduced solely through the contact constraint.

Contact introduces geometric couplings that are not aligned with the mesh topology, as nodes on one surface interact with geometrically nearby but topologically unrelated regions of the opposing surface. In sparsity-based formulations, such long-range interactions must be explicitly reflected in the construction of the sparsity pattern. In contrast, the matrix-free formulation introduced here incorporates contact directly through an additional contribution to the global energy functional, without requiring any modification of the underlying algebraic structure.

We enforce the contact constraint using a penalty formulation by augmenting the total energy functional with a contact term Ψ_c defined through a penalty density

$$\psi_c = \frac{1}{2} \kappa g(u_1, u_2)^2,$$

where g denotes the normal gap function and κ is the penalty parameter. The total energy functional then becomes

$$\Psi(u_1, u_2) = \int_{\Omega_{\text{sphere}}} \psi_\epsilon(u_1) \, d\Omega + \int_{\Omega_{\text{cube}}} \psi_\epsilon(u_2) \, d\Omega + \frac{1}{2} \int_{S_{\text{sphere}}} \psi_c(u_1, u_2) \, dS + \frac{1}{2} \int_{S_{\text{cube}}} \psi_c(u_1, u_2) \, dS. \quad (9)$$

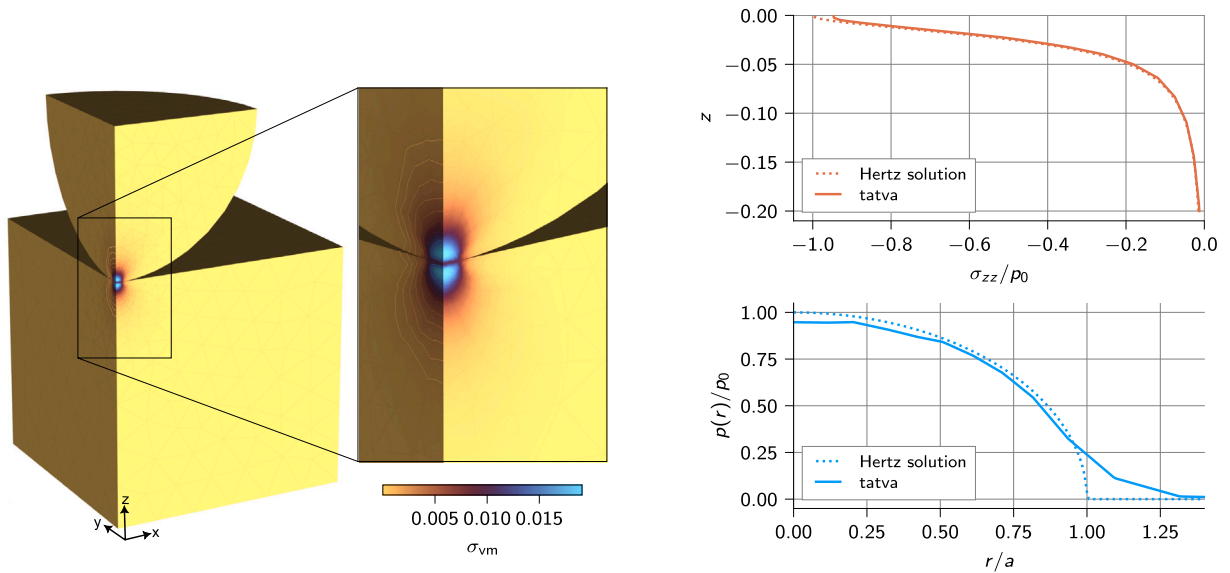


Fig. 8. Three-dimensional Hertzian contact benchmark. The sphere radius is $R_{\text{Hertz}} = 0.6$, and both bodies are assumed to behave linear elastically with identical material parameters $E = 1.0$ and $\nu = 0.3$ (left) Quarter-symmetry model of a spherical indenter in frictionless contact with an elastic half-space; the inset shows a magnified view of the contact region. The deformed configuration is colored by the von Mises stress, σ_{vm} . (right-top) Normalized subsurface stress component σ_{zz}/p_0 along the symmetry axis beneath the contact center, compared with the analytical Hertz solution. (right-bottom) Normalized contact pressure distribution $p(r)/p_0$ as a function of the radial coordinate r/a , where a is the contact radius and p_0 the maximum contact pressure. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

To evaluate the contact energy, we use a two-pass node-to-surface contact detection algorithm to determine the gap $g(\mathbf{u}_1, \mathbf{u}_2)$. In the first pass, we orthogonally project every surface node of the sphere onto the cube surface to compute the gap and then integrate the penalty density over the surface of the sphere. In the second pass, we compute the gap from every surface node of the cube to the sphere surface and then integrate the corresponding penalty energy over the surface of the cube. We define the total contact energy as the average of these two contributions:

```
def compute_contact_energy(x1, x2):
    g1 = ... # gap for sphere surface nodes using node-to-surface contact algorithm
    g2 = ... # gap for cube surface nodes using node-to-surface contact algorithm
    contact_energy_1 = ... # integrate
    contact_energy_2 = ... # integrate
    return 0.5*(contact_energy_1 + contact_energy_2)

def total_energy(z_flat: Array) -> float:
    u1, u2 = slice_and_reshape_z(z_flat)
    # [...] Compute elastic energies for both bodies -> elastic_energy_1, elastic_energy_2
    # Compute contact energy based on two bodies' current configurations
    contact_energy = compute_contact_energy(x1 + u1, x2 + u2)
    return (op_1.integrate(elastic_energy_1) + op_2.integrate(elastic_energy_2)
            + contact_energy)
```

Fig. 8 compares the numerical results with the analytical Hertz solution. Overall, we observe good agreement in both the subsurface stress distribution and the contact pressure profile. The deviations in the contact pressure outside the contact area are attributed to the relatively coarse discretization of the surface mesh.

We note that in this example, we deliberately employ a simple contact treatment to emphasize the generality of the energy-centric approach. More advanced contact methods, such as surface-to-surface or mortar methods [43], can be incorporated in the same manner by modifying the contact energy term. Because AD automatically handles the resulting cross-coupling terms automatically, the framework substantially simplifies the implementation of contact problems involving complex geometric interactions.

6.3. Mixed domain or mixed dimensional problems

Mixed-domain or mixed-dimensional problems involve interactions between physical processes defined on regions of different topological dimensions, such as cohesive fracture along a 2D interface embedded in a 3D domain. Simulating such systems in

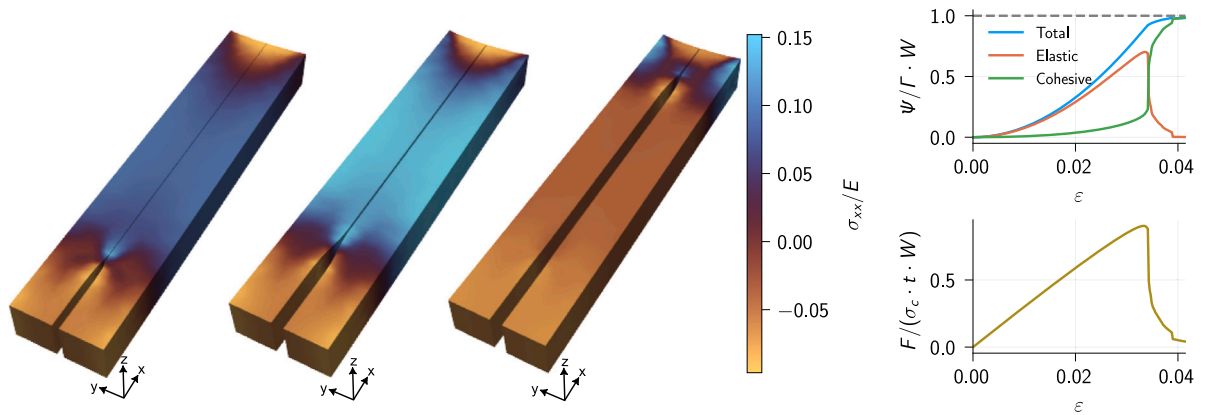


Fig. 9. Cohesive fracture along an interface under Mode-I loading in the y -direction. A linear elastic body of dimensions $10L \times 2L \times L$ with $L = 0.005$ and with an initial interfacial crack of length L is loaded under displacement control. (left) Deformed configuration at three loading steps, illustrating crack propagation along the interface. Colors indicate the normal stress component σ_{xx} , normalized by the Young's modulus. (right-top) Evolution of the total strain energy, total fracture energy, and the total energy during loading. The total dissipated energy matches the theoretical value $\Gamma \times A_{\text{crack}}$ required to fully separate the interface. (right-bottom) Reaction force at the top edge as a function of the applied strain, with the sudden force drop indicating brittle fracture behavior.

standard finite-element frameworks can be challenging [44], as it requires managing multiple incompatible meshes, interpolating solution fields across non-conforming dimensions (e.g., from 3D bulk elements to 2D surface elements), and constructing complex assembly mappings to couple the different domains.

In the proposed energy-centric framework, we address these challenges by treating the physics associated with each domain as an additive contribution to the global energy functional. Lower-dimensional contributions enter the formulation as additional energy terms defined on their respective geometries, while AD handles the resulting couplings consistently at the global level. This approach avoids the need for specialized assembly logic or explicit domain-to-domain mapping procedures. In the following examples, we illustrate how this formulation simplifies the implementation of mixed-dimensional topologies within a unified computational framework.

6.3.1. Fracture using cohesive elements

Cohesive fracture modeling typically relies on specialized interface elements [45,46], introducing a mixed-dimensional setting in which the bulk material is discretized with 3D volume elements while the fracture process zone is represented by 2D surface elements. In traditional finite-element frameworks, managing the connectivity between these topologically distinct domains and enforcing traction–separation relations requires dedicated mesh data structures and specialized element formulations.

In the proposed energy-centric framework, we treat the cohesive interface as an additional surface energy contribution to the global potential. The total energy of the system is expressed as the sum of the bulk energies of the two bodies, defined on domains Ω_a and Ω_b , and the fracture energy defined on the cohesive interface S_{coh} :

$$\Psi(\mathbf{u}) = \int_{\Omega_a} \psi_\epsilon(\mathbf{u}_a) \, d\Omega + \int_{\Omega_b} \psi_\epsilon(\mathbf{u}_b) \, d\Omega + \int_{S_{\text{coh}}} \psi_{\text{coh}}(\delta) \, dS, \quad (10)$$

where the interface potential ψ_{coh} depends on effective opening δ , defined as the norm of the displacement jump vector $\llbracket \mathbf{u} \rrbracket = \mathbf{u}_b - \mathbf{u}_a$ across the interface.

We employ an exponential cohesive law governed by the fracture energy Γ and the critical strength σ_c , with potential and traction response:

$$\psi(\delta) = \Gamma \left[1 - \left(1 + \frac{\delta}{\delta_c} \right) \exp \left(-\frac{\delta}{\delta_c} \right) \right], \quad T(\delta) = \frac{\Gamma}{\delta_c^2} \delta \exp \left(-\frac{\delta}{\delta_c} \right), \quad (11)$$

where the characteristic length is given by $\delta_c = (\Gamma \exp(-1))/\sigma_c$.

To ensure thermodynamic consistency, the fracture process must be irreversible. We enforce this condition directly at the level of the potential by tracking the maximum historical opening δ_{max} . During unloading, the material response follows a linear path to the origin rather than retracing the softening curve:

$$\psi_{\text{irrev}}(\delta) = \begin{cases} \psi(\delta) & \text{if } \delta = \delta_{\text{max}} \quad (\text{Loading}) \\ \psi(\delta_{\text{max}}) - \frac{1}{2}(\delta_{\text{max}} - \delta) \left(T(\delta_{\text{max}}) + \frac{T(\delta_{\text{max}})}{\delta_{\text{max}}} \delta \right) & \text{if } \delta < \delta_{\text{max}} \quad (\text{Unloading}) \end{cases} \quad (12)$$

We verify this formulation by simulating Mode-I fracture in a double cantilever beam under quasi-static loading, as shown in Fig. 9. We use the bulk material and interface parameters reported in [47]. We compute the displacement jump at the interface

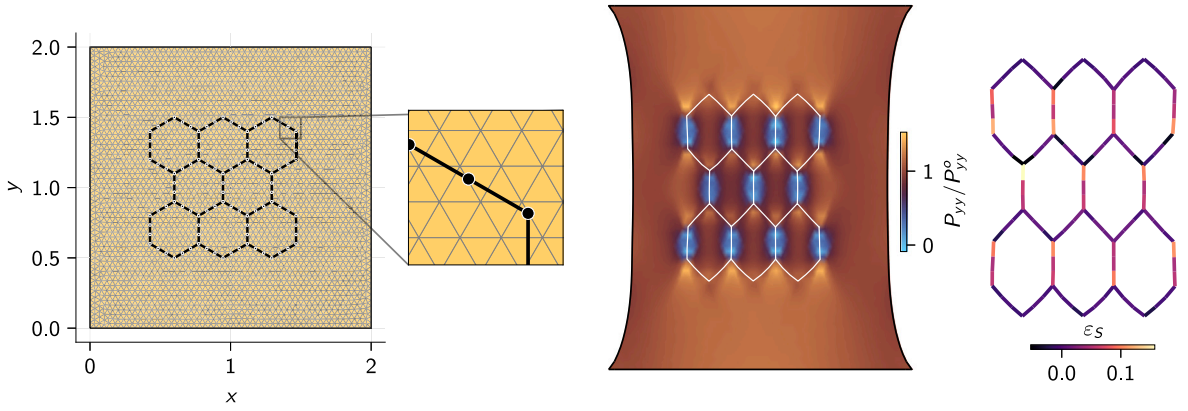


Fig. 10. Fiber-reinforced soft material under uniaxial tension. (left) Schematic of a hexagonal fiber network embedded in a compressible neo-Hookean material. The two domains are discretized non-conformingly, such that fiber nodes are embedded within triangular bulk elements (see inset). The fibers have cross-sectional area A_{fiber} and stiffness $E_{\text{fiber}} = 100E_{\text{bulk}}$. (middle) Deformed bulk configuration at an applied stretch of 1.2, colored by the first Piola–Kirchhoff stress component P_{yy} normalized by P_{yy}^o , which denotes the bulk stress under the same imposed deformation without fibers. (right) The deformed configuration of the embedded fiber network, colored by the strain ϵ_s in each fiber. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

nodes, evaluate it at the quadrature points, and integrate the resulting cohesive energy density along the interfaces. The following code illustrates the implementation of cohesive energy, including the enforcement of irreversibility:

```
def exponential_cohesive_energy(jump: Array, delta_max: float) -> float:
    delta = compute_opening(jump) # compute effective opening

    def opening_branch(delta):
        loading = lambda delta : exponential_potential(delta, Gamma, delta_c)

        def unloading(delta):
            ... # compute psi_irrev as in Equation 12
            return psi_irrev
        return jax.lax.cond(delta > delta_max, loading, unloading, delta)

    penetration = lambda delta : 0.5 * penalty * delta**2
    return jax.lax.cond(delta > 1e-8, opening_branch, penetration_branch, delta)

def total_cohesive_energy(u_flat: Array, delta_max: Array) -> float:
    u = u_flat.reshape(-1, n_dofs_per_node)
    jump = u.at[top_interface_nodes, :].get() - u.at[bottom_interface_nodes, :].get()
    jump_quad = op_quad.eval(jump) # Operator defined at the interface
    cohesive_energy_density = ... # call exponential_cohesive_energy
    return op_quad.integrate(cohesive_energy_density)
```

We solve the resulting nonlinear problem using a matrix-free Newton–Krylov method. As a validation, we compare the total dissipated energy with the theoretical value $\Gamma \times A_{\text{crack}}$ required to fully separate the interface. The close agreement observed in Fig. 9 confirms that the mixed-dimensional coupling correctly transmits forces and dissipates energy, without relying on specialized “interface element” classes.

6.3.2. Embedded fiber network in a soft material

As a second example of mixed-dimensional coupling, we consider a fiber-reinforced composite in which stiff 1D fibers are embedded in a soft 2D or 3D bulk material, see Fig. 10. Such configurations arise in biological tissues (e.g., muscle fibers) and engineering composites. A central difficulty in these problems is that the fiber geometry typically does not align with the bulk mesh. The fibers may intersect bulk elements at arbitrary positions and orientations.

We address this using an embedded-element approach formulated entirely at the level of the total potential energy. We treat the fiber and bulk contributions as separate energy terms. The bulk energy Ψ_{bulk} is integrated over the triangular elements, while the fiber energy Ψ_{fiber} is integrated along 1D line elements. The total energy is thus given as

$$\Psi(\mathbf{u}) = \underbrace{\int_{\Omega_{\text{bulk}}} \psi_{\epsilon}(\nabla \mathbf{u}) \, d\Omega}_{\Psi_{\text{bulk}}} + \underbrace{\int_{\Gamma_{\text{fiber}}} \psi_{\text{fiber}}(\epsilon_{\text{fiber}}) \, dS}_{\Psi_{\text{fiber}}}, \quad (13)$$

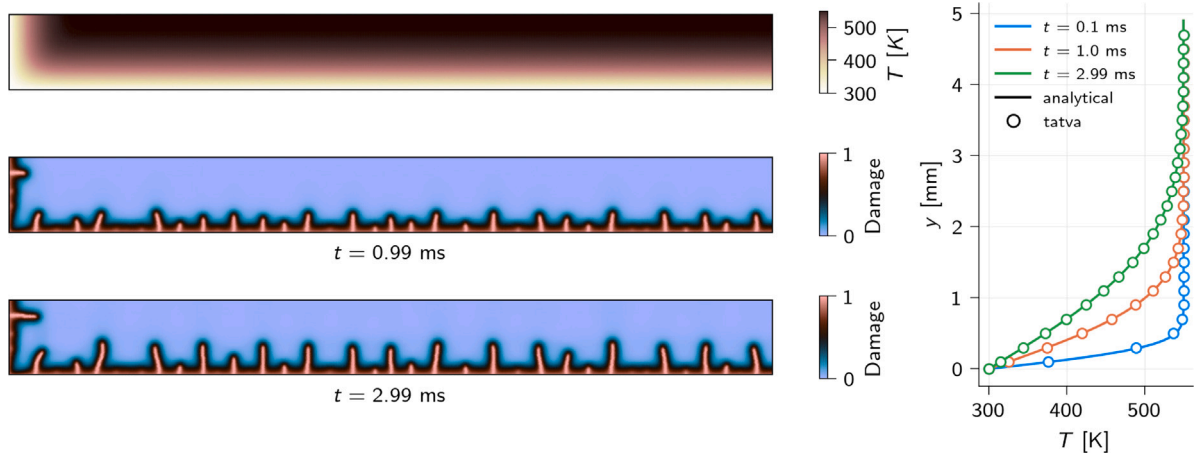


Fig. 11. Phase-field fracture during thermal shock of a ceramic slab (50×4.9 mm) initially at $T = 550$ K, subjected to sudden cooling at the boundaries to $T = 300$ K. The material and geometric parameters for the problem are adopted from [48]. (left-top) Temperature distribution at $t = 2.99$ ms and (left-middle-bottom) corresponding crack patterns at $t = 0.99$ ms and 2.99 ms. Cracks initiate at the cooled surface and propagate inward under thermal gradient, consistent with experimental findings [49]. (right) Temperature profile along the slab height at three time steps compared with the analytical solution [48], showing excellent agreement.

where we define the fiber strain energy density as $\psi_{\text{fiber}}(\epsilon_{\text{fiber}}) = \frac{1}{2} E_{\text{fiber}} A_{\text{fiber}} \epsilon_{\text{fiber}}^2$. In this example, we model the bulk as a compressible neo-Hookean material.

The key step is coupling the fiber kinematics to the bulk deformation. Because the meshes are non-conforming, we perform a geometric search prior to the simulation. For each fiber node, we identify the containing triangular bulk element and compute its barycentric coordinates within that element. During the simulation, we use this precomputed mapping to interpolate the bulk displacement field to the fiber nodes.² This procedure effectively “ties” the fiber to the surrounding bulk without introducing additional DoFs or modifying the mesh topology.

We then compute the fiber strain at the quadrature-points and the strain energy of the fiber based on this interpolated displacement as:

```
def fiber_strain_energy(u_flat: Array, barycenters: Array) -> float:
    u = u_flat.reshape(-1, n_dofs_per_node) # bulk displacements
    u_at_nodes = ... # project the bulk displacement to the fiber nodes using barycenters
    u_grad = op_line.grad(u_at_nodes)
    energy_density = compute_fiber_energy(u_grad)
    return op_line.integrate(energy_density)
```

We finally add this scalar contribution to the global energy functional. AD propagates the fiber stiffness consistently to the bulk DoFs through the chain rule. As a result, the framework handles non-conforming embedded reinforcements in a purely variational manner, without specialized element types or intrusive modifications to the discretization.

6.4. Multiphysical coupling

Multiphysics problems involve the interaction of multiple field variables within a single model. In a variational setting, these interactions can be expressed through a unified energy functional that aggregates the contributions of all physical processes. We demonstrate the capability of the proposed framework to handle such couplings by simulating thermal shock fracture in a 2D ceramic slab [49].

This problem involves a three-way coupling: the thermal field induces mechanical strains, the elastic energy drives fracture, and the evolving phase-field degrades the material stiffness.

² By completing the spatial mapping before the energy density computation loop, interpolating the displacements easily supports the element batching discussed in Section 2. The batching operations simply process the pre-mapped data without encountering the complex search logic.

At each time step, we minimize an incremental potential $\Psi(\mathbf{u}, d, T)$, which combines elastic-thermal energy, fracture surface energy, and a transient heat potential:

$$\begin{aligned} \Psi = & \underbrace{\int_{\Omega} [g(d)\psi_{\epsilon}^+(\mathbf{u}, \Delta T) + \psi_{\epsilon}^-(\mathbf{u}, \Delta T)] \, d\Omega}_{\text{Elastic-Thermal Energy}} + \underbrace{\int_{\Omega} \Gamma \left[\frac{(d-1)^2}{2l_0} + \frac{l_0}{2} |\nabla d|^2 \right] \, d\Omega}_{\text{Fracture Surface Energy}} \\ & + \underbrace{\int_{\Omega} \frac{1}{2} \Delta t (T^t - T^{t-1})^2 \, d\Omega + \int_{\Omega} \kappa \nabla T \cdot \nabla T \, d\Omega}_{\text{Transient Heat Potential } \Psi_{\text{th}}} . \end{aligned} \quad (14)$$

Here, ψ_{ϵ}^+ and ψ_{ϵ}^- denote the tensile and compressive parts of the elastic strain energy density [50], including thermal expansion effects. Further, $g(d) = (1-d)^2$ is the degradation function, Γ is the fracture energy, and l_0 is the phase-field length scale.

The thermal contribution, Ψ_{th} , corresponds to a backward Euler discretization of the transient heat equation and is implemented as

```
def thermal_energy_density(grad_T, kappa):
    return kappa * jnp.vdot(grad_T, grad_T)

def total_thermal_energy(T_new: Array, T_prev: Array, dt: float) -> float:
    T_new_quad = op.eval(T_new) # temperature at time t
    T_prev_quad = op.eval(T_prev) # temperature at time t-1
    grad_T = op.grad(T_new)
    diff_term = thermal_energy_density(grad_T, kappa)
    inertial_term = (0.5 / dt) * (T_new_quad - T_prev_quad)**2 # temporal
    return op.integrate(diff_term + inertial_term)
```

The elastic and fracture energy terms follow directly from the formulations introduced in earlier examples.

For this simulation, we employed a staggered (operator-split) solution strategy. At each time step, we sequentially minimize the potential with respect to the displacement field $\mathbf{u}$, the phase-field d , and the temperature field T , iterating until convergence.

As shown in Fig. 11, cracks initiate at the surface and propagate inward under the imposed thermal gradient, in qualitatively agreement with experimental observations [49]. Furthermore, the temperature distribution inside the slab exhibits excellent agreement with the corresponding analytical solution [48].

A key advantage of the energy-centric architecture is that switching from a staggered to a fully monolithic solution strategy requires no modifications to the physics implementation. To perform a monolithic solve, we take a similar approach as in Section 6.2.2 and concatenate all DoFs into a single global state vector $\mathbf{z} = [\mathbf{u}, d, T]$ and define the total energy directly in terms of this vector:

```
def total_energy(z_flat: Array) -> float:
    u, d, T = slice_and_reshape_z(z_flat)
    ... # compute strain energy density
    ... # compute fracture energy density
    ... # compute thermal potential density
    return op.integrate(strain_energy_density)
        + op.integrate(fracture_energy_density)
        + op.integrate(thermal_energy_density)

monolithic_residual = jax.jacrev(total_energy)
monolithic_tangent = jax.jvp(total_energy, (z_prev,), (dz,))[1]
```

AD then computes the full Hessian (or JVP) with respect to $\mathbf{z}$, automatically capturing all cross-coupling terms, such as $\nabla_{\mathbf{u}, T}^2 \Psi$, without manual block assembly.

6.5. Non-variational problem on non-Euclidean manifolds

To demonstrate that the framework is not limited to conservative systems, we solve an advection-diffusion problem for a scalar field c on a spherical surface S [51]. The governing equation reads

$$\frac{\partial c}{\partial t} + \mathbf{u} \cdot \nabla_s c = D \Delta_s c , \quad (15)$$

where D denotes the diffusivity, $\mathbf{u}$ is a velocity field tangent to the sphere, Δ_s is the Laplace–Beltrami operator, and ∇_s is the surface gradient.

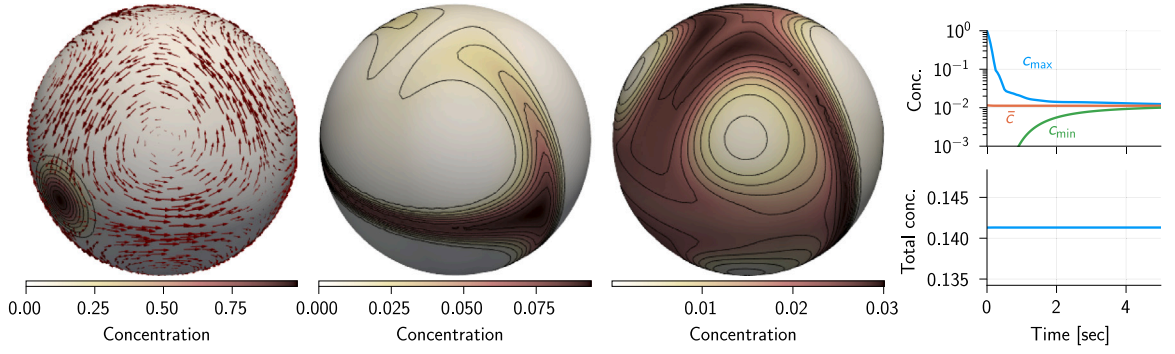


Fig. 12. Advection–diffusion of a scalar concentration on a spherical surface. (left) Evolution of the concentration field at three time steps. The initial Gaussian pulse advects under a divergence-free velocity field (red arrows scaled by the velocity magnitude) and gradually diffuses. (middle) Concentration profile at $t = 1$ s showing dominant advective transport followed by diffusion. (right-top) Evolution of maximum, mean, and minimum concentration values over time, illustrating homogenization due to diffusion. (right-bottom) Total surface concentration $\int c \, dS$ remains constant, confirming mass conservation and numerical robustness.

Unlike previous examples, the advective term makes the operator non-symmetric. Consequently, no scalar potential Ψ exists whose variation yields the governing equation. Instead, we formulate the problem using the principle of virtual work, as outlined in Section 3.1. The weak form is given as:

$$\int_S \frac{\partial c}{\partial t} v \, dS + \int_S (\mathbf{u} \cdot \nabla_s c) v \, dS + \int_S D \nabla_s c \cdot \nabla_s v \, dS = 0, \quad (16)$$

where v denotes the test function. For this example, we restrict the velocity field to be *divergence-free*, i.e. $\nabla_s \cdot \mathbf{u} = 0$, which simplified the weak formulation.

We implement this formulation using the same operators employed in the previous energy-based examples. No special plugins or problem-specific extensions are required. The scalar virtual-work functional is defined as

```
def diffusion_energy_density(grad_c, grad_v, D):
    return D * jnp.vdot(grad_c, grad_v)

def total_virtual_work(c_flat, v_flat):
    grad_c = op.grad(c_flat)
    grad_v = op.grad(v_flat)
    density = diffusion_energy_density(grad_c, grad_v, D)
    return op.integrate(density)

compute_internal_force = jax.jacrev(total_virtual_work, argnums=1)
residual = compute_internal_force(u_flat, v_flat=jnp.zeros(n_dofs))
```

AD then produces the residual vector and the action of the non-symmetric tangent operator within the same monolithic architecture.

We prescribe a velocity field corresponding to rigid-body rotation about the z -axis, $\mathbf{u} = \boldsymbol{\omega} \times \mathbf{x}$, which is analytically divergence-free on the sphere. Starting from a Gaussian concentration pulse (see Fig. 12-left), the solution first advects along the imposed velocity field and subsequently diffuses over the surface, see Fig. 12-middle. As shown in Fig. 12-right, the maximum and minimum concentrations converge toward the mean concentration over time, while the total concentration remains constant, confirming mass conservation.

This example highlights two key aspects of our framework. First, it supports non-variational and non-symmetric operators without modifying the architecture. Second, extending the formulation to a curved manifold requires only redefining the element-level Jacobian and gradient operators (see Appendix E). Once these geometric operators are provided, the same global energy/virtual-work structure and AD machinery apply unchanged.

6.6. Integration of data-driven approaches

As a final demonstration of the proposed framework's versatility, we integrate data-driven models directly into the energy-centric finite-element formulation. Because the proposed architecture operates entirely on scalar functionals and relies on AD, it allows neural networks and other differentiable models to be incorporated without modifying the solver or assembly logic.

6.6.1. Neural Constitutive Modeling (NCM)

NCMs offer a flexible way to represent complex material behavior beyond classical phenomenological laws. Recent works [14,25] have successfully integrated NCMs into finite-element simulations by coupling differentiable machine learning libraries with conventional solvers. However, these implementations are typically embedded within legacy, element-centric frameworks that rely on scatter-add assembly. As a result, while the neural network enhances constitutive expressivity, the overall architecture remains constrained by the underlying assembly workflow.

In the proposed framework, NCMs are not external components but native parts of the energy formulation. Because the entire framework is built on a differentiable infrastructure, a data-driven potential $\Psi_{NN}(\mathbf{F}; \theta)$ is treated exactly like a classical strain energy density (see Sections 6.1 and 6.2.1). At the quadrature level, we simply replace the analytical energy density with a neural network evaluation,

```
def strain_energy_density(grad_u):
    I = jnp.eye(3)
    F = I + grad_u
    C = F.T @ F # computing Cauchy strain
    ... # computing invariants
    return neural_network(I1, J) # calling neural network
```

The neural network maps strain invariants, such as $I_1 = \text{tr}(C)$ and $J = \det(F)$, to a scalar strain energy density. Since the total potential energy is defined globally, AD propagates derivatives through the neural network and into the residual and tangent operators without any additional implementation effort.

To illustrate this integration, we simulate a uniaxial tension test on a 3D hexagonal architected meta-material, see Fig. 13-left. At each quadrature point, we evaluate a feed-forward multilayer perceptron with two hidden layers and `softplus` activation to ensure twice differentiability of the energy. To stabilize the initial iterations, we regularize the neural energy density as

$$\psi_{NN}(I_1, J) = [\text{NN}(I_1, J; \theta) - \text{NN}(3, 1; \theta)] + \psi_{\text{base}}(I_1, J), \quad (17)$$

where the shift enforces zero energy in the reference configuration and the convex base energy density ensures a positive-definite tangent stiffness.

For demonstration purposes, we employ an untrained synthetic network. The Newton solver converges in five iterations and produces physically plausible displacements (see Fig. 13-left). This example highlights that neural constitutive models can be integrated seamlessly into the global energy formulation of our proposed framework, without modifying the solver, assembly logic, or differentiation pipeline.

We emphasize that the framework is not limited to multilayer perceptrons. Any differentiable architecture supported by the AD backend, including Convolutional Neural Networks (CNNs), input convex neural networks (ICNNs) [30], constitutive artificial neural networks (CANNs) [52], or input convex Kolmogorov-Arnold networks (ICKANs) [53], can be incorporated in the same manner.

6.6.2. Neural-Operator Element Method (NOEM)

While neural constitutive modeling replaces local material behavior at the quadrature level, the same energy-centric architecture also supports domain-level learning approaches such as the NOEM [33,54]. In NOEM, an entire subdomain, e.g., a region with intricate microstructure or geometric complexity, is replaced by a single neural element. Instead of discretizing this region explicitly, a pre-trained neural operator predicts its total potential energy from the displacement field prescribed on its boundary.

A key challenge in conventional implementations is enforcing displacement continuity at the interface and deriving the stiffness terms that couple the neural region to the surrounding finite-element mesh. In the proposed framework, this coupling follows directly from energy superposition. We express the global potential as

$$\Psi(\mathbf{u}) = \underbrace{\int_{\Omega_{\text{FEM}}} \psi_{\epsilon}(\nabla \mathbf{u}) \, d\Omega}_{\Psi_{\text{FEM}}} + \Psi_{\text{NO}}(\mathbf{u}_{\text{interface}}) - \Psi_{\text{ext}}. \quad (18)$$

Here, Ψ_{NO} is a scalar function that maps the interface displacements $\mathbf{u}_{\text{interface}}$ to the total energy of the neural inclusion. Because we define the neural contribution as an additive term in the global energy function, AD generates the interface forces and all cross-coupling stiffness terms without additional derivatives.

The implementation only requires extracting the interface displacements and passing them to the neural operator:

```
def total_energy(u_flat: Array) -> float:
    u = u_flat.reshape(-1, n_dofs_per_node)
    energy_fem_domain = total_strain_energy(u_flat) # same function in Listing 1
    u_interface = u[interface_idx] # extract displacements for interface nodes
    energy_neural_domain = neural_operator(u_interface) # psi_no

    return energy_fem_domain + energy_neural_domain
```

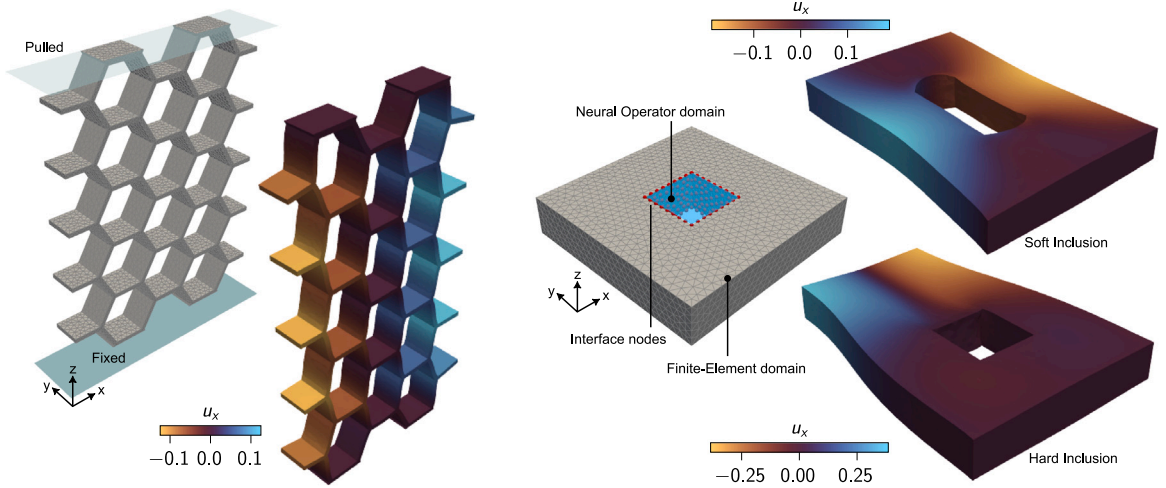


Fig. 13. Incorporating neural networks within the energy-centric framework. (left) Hexagonal meta-material under uniaxial tension applied in the z -direction. A neural constitutive model is evaluated at each quadrature point to predict strain energy density from strain invariants. The color map shows the displacement component in the x -direction, illustrating a physically plausible deformation response. (right) Neural-Operator Element Method for multiscale modeling. The blue region indicates the subdomain represented by a neural operator, while the red nodes denote the interface between the neural domain and the classical FE domain. Loading is applied in the y -direction. The upper and lower subfigures correspond to soft and stiff neural inclusions, respectively. The color maps show the displacement component in the x -direction. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

To demonstrate this formulation, we consider a 3D neural inclusion benchmark (see Fig. 13-right). We generate an unstructured tetrahedral mesh of a cuboid domain containing a central square cutout. We do not mesh the cutout region. Instead, we treat it as a single neural element whose response depends only on the interface displacements. For this demonstration, we use an untrained multilayer perceptron with random weights to verify that the solver correctly handles domain coupling and derivative propagation.

To establish a physically consistent reference state, we enforce zero energy at zero displacement by shifting the neural potential

$$\Psi_{\text{NO}}(\mathbf{u}_{\text{interface}}) = [\text{NN}(\mathbf{u}_{\text{interface}}; \theta) - \text{NN}(\mathbf{0}; \theta)] . \quad (19)$$

Because the neural inclusion contributes directly to the global energy, AD produces all interface forces and stiffness contributions consistently. The Newton–Raphson solver converges to a residual tolerance of 10^{-12} without manually deriving interface traction expressions or assembling dedicated coupling blocks.

Although we use an untrained synthetic network in this example, the same architecture applies to trained neural operators such as Fourier Neural Operator (FNO) [55] or DeepONet [33]. In practice, a trained operator can replace a geometrically complex domain while preserving a monolithic and fully differentiable formulation.

7. Discussion

In this work, we introduced an energy-centric finite-element framework that derives residual vectors and tangent operators by applying AD directly to a global potential functional. The examples in Section 6 demonstrate that a monolithic differentiable engine can handle complex physical systems efficiently. Across problems involving nonlinear materials, multi-point constraints, contact, mixed-dimensional couplings, multiphysics interactions, non-variational formulations, and data-driven models, the implementation followed the same structural pattern: define a scalar functional and differentiate it globally. No new assembly routines, element types, or coupling blocks were required. This illustrates that the versatility of the proposed framework does not arise from adding specialized modules, but from delegating derivative construction to AD at the level of the global energy. Furthermore, because the framework relies entirely on an AD backend, the formulation is inherently end-to-end differentiable. This allows for seamless differentiation with respect to material parameters, boundary conditions, or nodal geometry, opening highly promising perspectives for various forms of inverse problems, parameter identification, and shape optimization without the need to derive complex adjoint solvers.

By incorporating global AD, we shift the construction of residuals and tangent operators from manual assembly logic to the compiler. This enables two complementary strategies: JVP for matrix-free schemes and graph-coloring-based sparse differentiation for explicit matrix construction. Unlike conventional scatter-add assembly architectures, which remain element-centric, the proposed formulation operates directly on the global functional. As shown in Section 5, both strategies achieve linear scaling with respect to the number of DoFs and maintain stable throughput on GPU hardware. The absence of unstructured atomic scatter operations removes a major source of memory contention in traditional assembly workflows. Recently, methods based on sparse matrix routing

have been proposed to alleviate the memory contention caused by scatter-add operations [56], but this strategy, as also stated in [56], is inherently confined to problems that possess a variational structure.

A notable consequence of representing the system through a single global functional is a numerical duality between robustness and scalability. Because both sparse differentiation and matrix-free Jacobian-vector products derive from the same global energy, the choice of linearization strategy becomes a numerical decision rather than an architectural constraint. This flexibility contrasts with traditional element-centric architectures, where matrix-free and assembled formulations often require separate implementation pathways. While the matrix-free approach based on JVP is entirely physics-agnostic, sparse matrix assembly through graph coloring requires prior knowledge of the sparsity pattern. For standard mechanics and saddle-point problems, this pattern is determined by mesh connectivity and is therefore known *a priori*, as shown in Appendix D. However, for problems with evolving connectivity, such as non-local interactions, identifying the sparsity pattern in advance becomes more challenging. Future work will focus on detecting sparsity patterns directly from the global energy functional, for example, through structured perturbations or randomized probing techniques. In addition, more advanced coloring strategies, including star or acyclic coloring, may further reduce the number of required evaluations for sparse assembly [27].

Regarding solver integration, the present framework focuses on residual and tangent construction while remaining fully compatible with existing solver libraries. The output of AD is either a sparse matrix or a linear operator, which can be passed directly to external libraries such as PETSc [57] or algebraic multigrid solvers. As a result, the proposed architecture does not require rewriting established linear or nonlinear solver infrastructures.

We also observed practical memory limits when reconstructing large, sparse stiffness matrices on a single device. On a single GPU, sparse differentiation became constrained at approximately five million DoFs. This limitation originates from the growth of the compiler's internal representation of the computational graph rather than from the algorithmic complexity of the formulation itself. As the problem size increases, the graph eventually exceeds internal serialization limits. An approach to overcome this limitation can be multi-device execution, which our energy-centric structure provides a natural pathway for. Because the global functional is expressed as a sum of independent element contributions, the computation is inherently suitable for data-parallel distribution. By partitioning the mesh and sharding state vectors across multiple accelerators, each device only processes a local portion of the computational graph. This strategy reduces per-device memory pressure while preserving linear scaling. Investigating distributed execution through mechanisms such as `jax.sharding` remains an important direction for future work.

These results indicate that finite-element formulations can be expressed directly at the level of the global variational problem without incurring the dense-operator overhead of naive global differentiation. By combining sparse differentiation and matrix-free Jacobian-vector products within a single architecture, the framework removes the reliance on scatter-add element assembly while preserving strong scaling behavior on modern accelerators. More broadly, the framework shifts the implementation effort from hand-written derivative and assembly kernels to the level of a global functional and its differentiation pipeline.

These observations suggest that monolithic, compiler-accelerated formulations provide a viable alternative to traditional assembly-based finite-element architectures for large-scale computational mechanics. Rather than optimizing legacy element-centric workflows for new hardware, the proposed approach redefines the representation of the global problem itself, aligning mathematical abstraction and computational execution within a single differentiable framework.

8. Conclusion

In this work, we resolve a persistent limitation in AD-based finite-element methods: the trade-off between high-level expressiveness and computational scalability. Previous approaches forced a choice between globally differentiated formulations with clear variational structure but prohibitive $\mathcal{O}(N^2)$ complexity, and element-wise assembly schemes that scale efficiently but constrain how complex physics can be expressed.

Our proposed framework reconciles these extremes. By formulating problems strictly at the level of a global energy functional, we preserve the full expressive power of variational mechanics, making it straightforward to represent diverse problems, ranging from mixed-dimensional coupling and multiphysics, to non-conservative systems and data-driven models. At the same time, we avoid the computational penalties traditionally associated with global differentiation. Through graph-coloring-based sparse differentiation and matrix-free Jacobian-vector products, we achieve linear $\mathcal{O}(N)$ scaling while maintaining compatibility with modern accelerator hardware. Unlike traditional scatter-add assembly, our approach eliminates atomic contention and sustains constant throughput on GPUs and TPUs.

We provide this framework as an open-source library, `tatva` [24]. More broadly, this work demonstrates that global AD and large-scale performance are not mutually exclusive. By delegating assembly and differentiation to the compiler, we decouple physical modeling from low-level implementation details. This shift enables a more versatile, scalable, and future-proof foundation for computational mechanics.

CRediT authorship contribution statement

Mohit Pundir: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Flavio Lorez:** Writing – original draft, Visualization, Software, Methodology, Investigation, Formal analysis. **David S. Kammer:** Writing – review & editing, Supervision, Funding acquisition, Conceptualization.

Code availability

All examples used in this paper are available in the *tatva* documentation [38]. The library *tatva* is released as an open-source project [24].

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

DSK and MP acknowledge support from the Swiss National Science Foundation under the SNSF, Switzerland starting grant (TMSGI2_211655). All authors acknowledge support from the ETH Research, Switzerland Grant (24-1 ETH-020). The authors also thank Aryan Sinha for useful discussions on neural networks and Daniel Rayneau-Kirkhope for his feedback on the manuscript. The authors also acknowledge the use of the ETH Zurich Euler cluster for performing the computational simulations in this work.

Appendix A. Coloring approaches for FEM

In this work, we use the distance-2 greedy algorithm to color a tangent stiffness matrix. The coloring algorithm makes use of the sparsity pattern to identify and assign same color to columns whose non-zero entries do not share a row. The maximum number of colors required to color a matrix is governed by the mesh connectivity and the element type. For 2D element types with local support, such as 3-node triangular elements or 4-node quadrilateral elements, the maximum number of colors needed typically lies in the range of 30–40, regardless of the number of DoFs, as illustrated in Fig. 14. For 3D element types such as 4-node tetrahedral elements or 8-node hexahedron elements, typically about 80–90 colors are needed. This is because a single node in a 3D mesh is connected to more elements than in a 2D mesh. As illustrated in Fig. 14, these upper bounds are essentially independent of the overall problem size.

Appendix B. Performance on various GPUs

See Fig. 15.

Appendix C. Effect of batch size on performance

We implemented element batching primarily to fit intermediate data structures into the available GPU memory. We chose a constant batch size ($N = 50,000$ in Section 5) that comfortably accommodates the memory requirements for both 2D and 3D problems within our hardware limits. While the batch size dictates how many elements the GPU processes simultaneously, increasing this size only improves computational throughput up to a certain threshold (as shown in Fig. 16). Once the batch size is large enough to achieve full hardware occupancy, meaning all processors and threads are fully engaged, further increases do not yield additional performance gains. Instead, the throughput saturates, and larger batches only increase peak memory consumption.

Appendix D. Constructing sparsity pattern

A sparsity pattern for a problem can be constructed in various ways. For single physical field problems, the standard approach is to use the mesh connectivity to construct the sparse pattern. Depending on the support (local or non-local) of the element type, one can detect which entries of the tangent stiffness matrix will be non-zero. For a Lagrangian functional $\mathcal{L}(\mathbf{u}, \lambda) = \Psi(\mathbf{u}) + \lambda \cdot \mathbf{g}(\mathbf{u})$ such as in Section 6.2.2 where we enforce constraints $\mathbf{g}(\mathbf{u})$ via Lagrange multipliers, the tangent stiffness matrix has a characteristic block structure:

$$\begin{bmatrix} \mathbf{K} & \mathbf{B}^T \\ \mathbf{B} & \mathbf{0} \end{bmatrix} \quad (20)$$

where $\mathbf{B}$ is given as

$$\mathbf{B} = \frac{\partial \mathbf{g}}{\partial \mathbf{u}}.$$

We see that the sparsity pattern of the above system is the union of the sparsity patterns of the matrices $\mathbf{K}$ and $\mathbf{B}$. The sparsity pattern of $\mathbf{K}$ can be constructed from mesh connectivity, while the sparsity pattern of $\mathbf{B}$ is computed using AD of the $\mathbf{g}(\mathbf{u})$ function as

```
B = jax.jacrev(g)(jnp.zeros(n_dofs))
```

Thus, using these two sparsity patterns, one can construct the sparsity pattern for a Lagrangian functional.

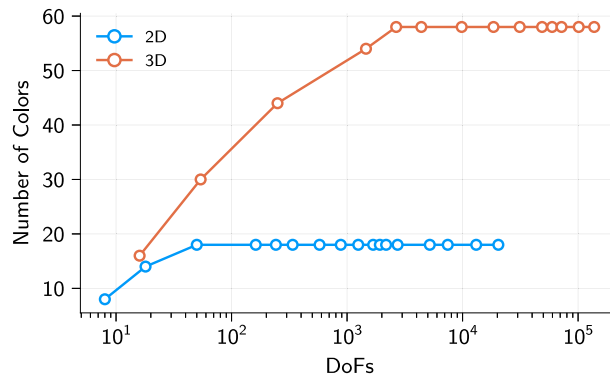


Fig. 14. Number of colors as a function of total degrees of freedom for a 2D and a 3D mesh with 2 degrees of freedom per node. The element type for 2D is a 3-node triangle, and for 3D, it is a 4-node tetrahedral element. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

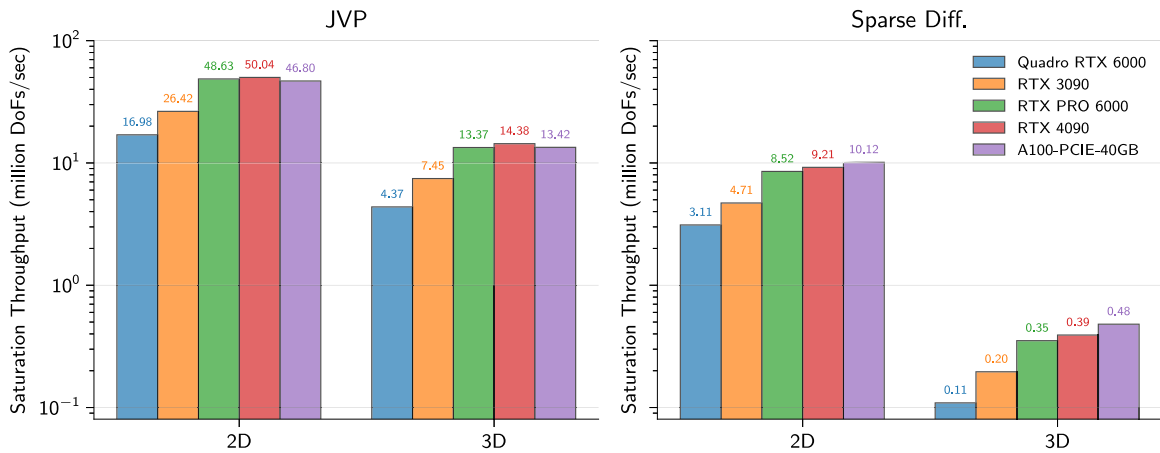


Fig. 15. Performance analysis of computing JVPs and sparse stiffness matrix via coloring approach on various GPUs. Here, we only show the saturated throughput values. The simulation setup is same as in Section 5. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

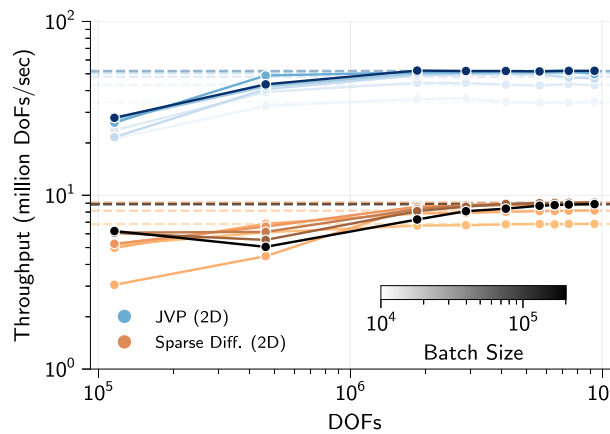


Fig. 16. The effect of various batch sizes on the efficiency of computing JVPs and computing sparse stiffness matrix via coloring approach. The results are shown for 2D setup as used in Section 5. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Appendix E. Implementation of triangular element in 3D space

For the implementation of a 2D triangular element in 3D space, we simply redefine the Jacobian calculation and the gradient calculation to enable surface gradients in 3D space.

```
class Tri3Manifold(element.Tri3):
    """A 3-node linear triangular element on a 2D manifold embedded in 3D space."""

    def get_jacobian(self, xi: Array, nodal_coords: Array) -> tuple[Array, Array]:
        dNdr = self.shape_function_derivative(xi)
        J = dNdr @ nodal_coords # shape (2, 2) or (2, 3)
        G = J @ J.T # shape (2, 2)
        detJ = safe_sqrt(jnp.linalg.det(G))
        return J, detJ

    def gradient(self, xi: Array, nodal_values: Array, nodal_coords: Array) -> Array:
        dNdr = self.shape_function_derivative(xi) # shape (2, 3)
        J, _ = self.get_jacobian(xi, nodal_coords) # shape (2, 3)
        G_inv = jnp.linalg.inv(J @ J.T) # shape (2, 2)
        J_plus = J.T @ G_inv # shape (3, 2)
        dudxi = dNdr @ nodal_values # shape (2, n_values)
        return J_plus @ dudxi # shape (3, n_values)
```

Data availability

We have added the weblink to the code in the manuscript.

References

- [1] O.C. Zienkiewicz, Robert L. Taylor, *The Finite Element Method*, fifth ed., Butterworth-Heinemann, Oxford ; Boston, ISBN: 978-0-7506-5049-6, 2000, 978-0-7506-5055-7 978-0-7506-5050-2.
- [2] Klaus-Jürgen Bathe, *Finite Element Procedures*, second ed., Prentice Hall, Englewood Cliffs, New Jersey, ISBN: 978-0-9790049-5-7, 1996.
- [3] Daniele Boffi, Franco Brezzi, Michel Fortin, *Mixed Finite Element Methods and Applications*, in: Springer Series in Computational Mathematics, vol. 44, Springer, Berlin, Heidelberg, ISBN: 978-3-642-36518-8, 2013, <http://dx.doi.org/10.1007/978-3-642-36519-5>, 978-3-642-36519-5. URL <https://link.springer.com/10.1007/978-3-642-36519-5>.
- [4] Igor A. Baratta, Joseph P. Dean, Jørgen S. Dokken, Michal Habera, Jack S. Hale, Chris N. Richardson, Marie E. Rognes, Matthew W. Scroggs, Nathan Sime, Garth N. Wells, *DOLFINx: the next generation FEniCS problem solving environment*, 2023, Published: preprint.
- [5] Jože Korelc, Peter Wriggers, *Automation of Finite Element Methods*, Springer International Publishing, Cham, ISBN: 978-3-319-39003-1, 2016, <http://dx.doi.org/10.1007/978-3-319-39005-5>, 978-3-319-39005-5. URL <http://link.springer.com/10.1007/978-3-319-39005-5>.
- [6] Charles C. Margossian, A review of automatic differentiation and its efficient implementation, *WIREs Data Min. Knowl. Discov.* (ISSN: 1942-4795) 9 (4) (2019) e1305, <http://dx.doi.org/10.1002/widm.1305>, URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/widm.1305>.
- [7] Atılım Güneş Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, Jeffrey Mark Siskind, Automatic differentiation in machine learning: a survey, *J. Mach. Learn. Res.* (ISSN: 1532-4435) 18 (1) (2017) 5595–5637, URL <https://dl.acm.org/doi/10.5555/3122009.3242010>.
- [8] Steffen Rothe, Stefan Hartmann, Automatic differentiation for stress and consistent tangent computation, *Arch. Appl. Mech.* (ISSN: 1432-0681) 85 (8) (2015) 1103–1125, <http://dx.doi.org/10.1007/s00419-014-0939-6>.
- [9] Samuel S. Schoenholz, Ekin D. Cubuk, JAX, M.D.: A framework for differentiable physics, 2020, URL <http://arxiv.org/abs/1912.04232>. arXiv:1912.04232 [cond-mat, physics:physics, stat].
- [10] Manuel Carrer, Henrique Musselli Cezar, Sigbjørn Løland Bore, Morten Ledum, Michele Cascella, Learning force field parameters from differentiable particle-field molecular dynamics, *J. Chem. Inf. Model.* (ISSN: 1549-9596) (2024) <http://dx.doi.org/10.1021/acs.jcim.4c00564>.
- [11] Artur P. Toshev, Harish Ramachandran, Jonas A. Erbesdobler, Gianluca Galletti, Johannes Brandstetter, Nikolaus A. Adams, JAX-SPH: A differentiable smoothed particle hydrodynamics framework, 2024, URL <http://arxiv.org/abs/2403.04750>. arXiv:2403.04750 [physics].
- [12] Tianju Xue, Shuheng Liao, Zhengtao Gan, Chanwook Park, Xiaoyu Xie, Wing Kam Liu, Jian Cao, JAX-FEM: A differentiable GPU-accelerated 3D finite element solver for automatic inverse design and mechanistic data science, *Comput. Phys. Comm.* (ISSN: 0010-4655) 291 (2023) 108802, <http://dx.doi.org/10.1016/j.cpc.2023.108802>, URL <https://www.sciencedirect.com/science/article/pii/S0010465523001479>.
- [13] Jeremy Bleyer, Numerical tours of computational mechanics with FEniCSx, 2024, URL <https://doi.org/10.5281/zenodo.10470942>.
- [14] Andrey Latyshev, Jérémy Bleyer, Corrado Maurini, Jack Hale, Expressing general constitutive models in FEniCSx using external operators and algorithmic automatic differentiation, *J. Theor. Comput. Appl. Mech.* (ISSN: 2726-6141) (2025) <http://dx.doi.org/10.46298/jtcam.14449>, URL <https://jtcam.episciences.org/16616>.
- [15] Tom Gustafsson, G.D. McBain, Scikit-fem: A Python package for finite element assembly, *J. Open Source Softw.* (ISSN: 2475-9066) 5 (52) (2020) 2369, <http://dx.doi.org/10.21105/joss.02369>, URL <https://joss.theoj.org/papers/10.21105/joss.02369>.
- [16] Andrea Vigliotti, Ferdinando Auricchio, Automatic differentiation for solid mechanics, *Arch. Comput. Methods Eng.* (ISSN: 1886-1784) 28 (3) (2021) 875–895, <http://dx.doi.org/10.1007/s11831-019-09396-y>.
- [17] Gaoyuan Wu, A framework for structural shape optimization based on automatic differentiation, the adjoint method and accelerated linear algebra, *Struct. Multidiscip. Optim.* (ISSN: 1615-147X) 66 (7) (2023) 151, <http://dx.doi.org/10.1007/s00158-023-03601-0>, 1615-1488. URL <http://arxiv.org/abs/2211.15409>. arXiv:2211.15409 [cs, math].
- [18] Naruki Ichihara, Naruki-Ichihara/feax, 2026, URL <https://github.com/Naruki-Ichihara/feax>. original-date: 2025-07-09T05:45:18Z.

- [19] Anders Logg, Kristian B. Ølgaard, Marie E. Rognes, Garth N. Wells, FFC: the FEniCS form compiler, in: Anders Logg, Kent-Andre Mardal, Garth N. Wells (Eds.), *Automated Solution of Differential Equations by the Finite Element Method*, in: *Lecture Notes in Computational Science and Engineering*, vol. 84, Springer, 2012, Section: 11.
- [20] Robert Anderson, Julian Andrej, Andrew Barker, Jamie Bramwell, Jean-Sylvain Camier, Jakub Cerveny, Veselin Dobrev, Yohann Doudout, Aaron Fisher, Tzanio Kolev, Will Pazner, Mark Stowell, Vladimir Tomov, Ido Akkerman, Johann Dahm, David Medina, Stefano Zampini, MFEM: A modular finite element methods library, *Comput. Math. Appl.* (ISSN: 0898-1221) 81 (2021) 42–74, <http://dx.doi.org/10.1016/j.camwa.2020.06.009>, URL <https://www.sciencedirect.com/science/article/pii/S0898122120302583>.
- [21] Łukasz Kaczmarczyk, Zahur Ullah, Karol Lewandowski, Xuan Meng, Xiao-Yi Zhou, Ignatios Athanasiadis, Hoang Nguyen, Christophe-Alexandre Chalons-Mouriesse, Euan J. Richardson, Euan Miur, Andrei G. Shvarts, Mebratu Wakeni, Chris J. Pearce, MoFEM: An open source, parallel finite element library, *J. Open Source Softw.* (ISSN: 2475-9066) 5 (45) (2020) 1441, <http://dx.doi.org/10.21105/joss.01441>, URL <https://joss.theoj.org/papers/10.21105/joss.01441>.
- [22] Nicolas Richart, Guillaume Anciaux, Emil Gallyamov, Lucas Frérot, David Kammer, Mohit Pundir, Marco Vocialta, Aurelia Cuba Ramos, Mauro Corrado, Philip Müller, Fabian Barras, Shenghan Zhang, Roxane Ferry, Shad Durussel, Jean-François Molinari, Akantu: an HPC finite-element library for contact and dynamic fracture simulations, *J. Open Source Softw.* (ISSN: 2475-9066) 9 (94) (2024) 5253, <http://dx.doi.org/10.21105/joss.05253>, URL <https://joss.theoj.org/papers/10.21105/joss.05253>.
- [23] Julian Andrej, Tzanio Kolev, Boyan Lazarov, Scalable analysis and design using automatic differentiation, 2025, URL <http://arxiv.org/abs/2506.00746>, arXiv:2506.00746 [math].
- [24] M. Pundir, F. Lorez, D.S. Kammer, Tatva: An energy-centric automatic differentiation framework for finite-element analysis (v0.9.1), 2026, <https://github.com/smec-ethz/tatva/tree/v0.9.1>. Open-source software.
- [25] Benjamin Alheit, Mathias Peirlinck, Siddhant Kumar, COMMET: Orders-of-magnitude speed-up in finite element method via batch-vectorized neural constitutive updates, *Comput. Methods Appl. Mech. Engrg.* (ISSN: 0045-7825) 452 (2026) 118728, <http://dx.doi.org/10.1016/j.cma.2026.118728>, URL <https://www.sciencedirect.com/science/article/pii/S0045782526000022>.
- [26] Mohit Pundir, David S. Kammer, Simplifying FFT-based methods for solid mechanics with automatic differentiation, *Comput. Methods Appl. Mech. Engrg.* (ISSN: 0045-7825) 435 (2025) 117572, <http://dx.doi.org/10.1016/j.cma.2024.117572>, URL <https://www.sciencedirect.com/science/article/pii/S0045782524008260>.
- [27] Alexis Montois, Guillaume Dalle, Assefaw Gebremedhin, Revisiting sparse matrix coloring and bicoloring, 2025, URL <http://arxiv.org/abs/2505.07308>, arXiv:2505.07308 [math].
- [28] Assefaw Hadish Gebremedhin, Fredrik Manne, Alex Pothen, What color is your Jacobian? Graph coloring for computing derivatives, *SIAM Rev.* (ISSN: 0036-1445) 47 (4) (2005) 629–705, <http://dx.doi.org/10.1137/S0036144504444711>, URL <https://epubs.siam.org/doi/10.1137/S0036144504444711>. Publisher: Society for Industrial and Applied Mathematics.
- [29] A.R. Curtis, M.J.D. Powell, J.K. Reid, On the estimation of sparse Jacobian matrices, *IMA J. Appl. Math.* (ISSN: 0272-4960) 13 (1) (1974) 117–119, <http://dx.doi.org/10.1093/imamat/13.1.117>.
- [30] Prakash Thakolkaran, Akshay Joshi, Yiwen Zheng, Moritz Flaschel, Laura De Lorenzis, Siddhant Kumar, NN-EUCLID: Deep-learning hyperelasticity without stress data, *J. Mech. Phys. Solids* (ISSN: 0022-5096) 169 (2022) 105076, <http://dx.doi.org/10.1016/j.jmps.2022.105076>, URL <https://www.sciencedirect.com/science/article/pii/S0022509622002538>.
- [31] Akshay Joshi, Prakash Thakolkaran, Yiwen Zheng, Maxime Escande, Moritz Flaschel, Laura De Lorenzis, Siddhant Kumar, Bayesian-EUCLID: Discovering hyperelastic material laws with uncertainties, *Comput. Methods Appl. Mech. Engrg.* (ISSN: 0045-7825) 398 (2022) 115225, <http://dx.doi.org/10.1016/j.cma.2022.115225>, URL <https://www.sciencedirect.com/science/article/pii/S0045782522003681>.
- [32] Zhichao Han, Mohit Pundir, Olga Fink, David S. Kammer, Learning physics-consistent material behavior from dynamic displacements, *Comput. Methods Appl. Mech. Engrg.* (ISSN: 0045-7825) 443 (2025) 118040, <http://dx.doi.org/10.1016/j.cma.2025.118040>, URL <https://www.sciencedirect.com/science/article/pii/S0045782525003123>.
- [33] Wei Wang, Maryam Hakimzadeh, Haihui Ruan, Somdatta Goswami, Accelerating multiscale modeling with hybrid solvers: Coupling FEM and neural operators with domain decomposition, 2025, URL <http://arxiv.org/abs/2504.11383>, arXiv:2504.11383 [cs] version: 2.
- [34] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, Qiao Zhang, JAX: composable transformations of Python+NumPy programs, 2018, URL <http://github.com/google/jax>.
- [35] Patrick Kidger, Cristian Garcia, Equinox: neural networks in JAX via callable PyTrees and filtered transformations, 2021, URL <http://arxiv.org/abs/2111.00254>, arXiv:2111.00254 [cs].
- [36] Mathis Gerdes, Mathisgerdes/jax-autovmap, 2025, URL <https://github.com/mathisgerdes/jax-autovmap>. original-date: 2022-05-25T10:56:08Z.
- [37] openxla/xla: A machine learning compiler for GPUs, CPUs, and ML accelerators. URL <https://github.com/openxla/xla>.
- [38] M. Pundir, F. Lorez, Tatva documentation and reproducible examples, 2026, <https://smec-ethz.github.io/tatva-docs/>. Documentation and executable examples (Google Colab).
- [39] Ted Belytschko, The finite element method: Linear static and dynamic finite element analysis: Thomas J. R. Hughes, *Comput.-Aided Civ. Infrastruct. Eng.* (ISSN: 10939687) 4 (3) (2008) 245–246, <http://dx.doi.org/10.1111/j.1467-8667.1989.tb00025.x>, URL <http://doi.wiley.com/10.1111/j.1467-8667.1989.tb00025.x>.
- [40] Ernst Gustav Kirsch, Die theorie der elastizität und die begründungen der festigkeitslehre, *Zeitschrift Vereines Dtsch. Ingenieure* 42 (1898) 797–807.
- [41] Jeremy Bleyer, Numerical tours of computational mechanics with FEniCS, 2018.
- [42] Kenneth Langstreth Johnson, *Contact Mechanics*, Cambridge University Press, 1987.
- [43] Alexander Popp, Mortar methods for computational contact mechanics and general interface problems, 2012, (September):34. URL <http://mediatum.ub.tum.de/?id=1109994>.
- [44] Cécile Daversin-Catty, Chris N. Richardson, Ada J. Ellingsrud, Marie E. Rognes, Abstractions and automated algorithms for mixed domain finite element methods, *ACM Trans. Math. Software* (ISSN: 0098-3500) 47 (4) (2021) 31:1–31:36, <http://dx.doi.org/10.1145/3471138>, URL <https://dl.acm.org/doi/10.1145/3471138>.
- [45] M. Ortiz, A. Pandolfi, Finite-deformation irreversible cohesive elements for three-dimensional crack-propagation analysis, *Internat. J. Numer. Methods Engrg.* (ISSN: 0029-5981) 44 (9) (1999) 1267–1282, [http://dx.doi.org/10.1002/\(SICI\)1097-0207\(19990330\)44:9<1267::AID-NME486>3.0.CO;2-7](http://dx.doi.org/10.1002/(SICI)1097-0207(19990330)44:9<1267::AID-NME486>3.0.CO;2-7), URL <http://doi.wiley.com/10.1002/%28SICI%291097-0207%2819990330%2944%3A9%3C1267%3A%3AAID-NME486%3E3.0.CO%3B2-7>. ISBN: 1097-0207.
- [46] M. Vocialta, N. Richart, J.-F.F. Molinari, 3D dynamic fragmentation with parallel dynamic insertion of cohesive elements, *Internat. J. Numer. Methods Engrg.* (ISSN: 10970207) 109 (12) (2017) 1655–1678, <http://dx.doi.org/10.1002/nme.5339>, URL <http://doi.wiley.com/10.1002/nme.5339>.
- [47] Mohit Pundir, Mokhtar Adda-Bedia, David S. Kammer, Transonic and supershear crack propagation driven by geometric nonlinearities, *Phys. Rev. Lett.* (ISSN: 0031-9007) 132 (22) (2024) 226102, <http://dx.doi.org/10.1103/PhysRevLett.132.226102>, 1079-7114. URL <https://link.aps.org/doi/10.1103/PhysRevLett.132.226102>.
- [48] Tushar Kanti Mandal, Vinh Phu Nguyen, Jian-Ying Wu, Chi Nguyen-Thanh, Alban de Vaucorbeil, Fracture of thermo-elastic solids: Phase-field modeling and new results with an efficient monolithic solver, *Comput. Methods Appl. Mech. Engrg.* (ISSN: 0045-7825) 376 (2021) 113648, <http://dx.doi.org/10.1016/j.cma.2020.113648>, URL <https://www.sciencedirect.com/science/article/pii/S0045782520308331>.
- [49] Yingfeng Shao, Yue Zhang, Xianghong Xu, Zhiliang Zhou, Wei Li, Boyang Liu, Effect of crack pattern on the residual strength of ceramics after quenching, *J. Am. Ceram. Soc.* (ISSN: 1551-2916) 94 (9) (2011) 2804–2807, <http://dx.doi.org/10.1111/j.1551-2916.2011.04728.x>, URL <https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1551-2916.2011.04728.x>, eprint: <https://ceramics.onlinelibrary.wiley.com/doi/pdf/10.1111/j.1551-2916.2011.04728.x>.

- [50] Marreddy Ambati, Tymofiy Gerasimov, Laura De Lorenzis, A review on phase-field models of brittle fracture and a new fast hybrid formulation, *Comput. Mech.* (ISSN: 01787675) 55 (2) (2014) 383–405, <http://dx.doi.org/10.1007/s00466-014-1109-y>, ISBN: 0178-7675.
- [51] M.E. Rognes, D.A. Ham, C.J. Cotter, A.T.T. McRae, Automating the solution of PDEs on the sphere and other manifolds in FEniCS 1.2, *Geosci. Model. Dev.* (ISSN: 1991-959X) 6 (6) (2013) 2099–2119, <http://dx.doi.org/10.5194/gmd-6-2099-2013>, URL <https://gmd.copernicus.org/articles/6/2099/2013/>.
- [52] Prakash Thakolkaran, Yaqi Guo, Shivam Saini, Mathias Peirlinck, Benjamin Alheit, Siddhant Kumar, Can KAN CANs? Input-convex Kolmogorov-Arnold Networks (KANs) as hyperelastic constitutive artificial neural networks (CANs), *Comput. Methods Appl. Mech. Engrg.* (ISSN: 0045-7825) 443 (2025) 118089, <http://dx.doi.org/10.1016/j.cma.2025.118089>, URL <https://www.sciencedirect.com/science/article/pii/S0045782525003615>.
- [53] Mathias Peirlinck, Kevin Linka, Juan A. Hurtado, Ellen Kuhl, On automated model discovery and a universal material subroutine for hyperelastic materials, *Comput. Methods Appl. Mech. Engrg.* (ISSN: 0045-7825) 418 (2024) 116534, <http://dx.doi.org/10.1016/j.cma.2023.116534>, URL <https://www.sciencedirect.com/science/article/pii/S0045782523006588>.
- [54] Wei Wang, Maryam Hakimzadeh, Haihui Ruan, Somdatta Goswami, Time marching neural operator FE coupling: AI accelerated physics modeling, 2025, URL <http://arxiv.org/abs/2504.11383>. [arXiv:2504.11383](https://arxiv.org/abs/2504.11383) [cs].
- [55] Binh Huy Nguyen, Matti Schneider, Universal Fourier neural operators for micromechanics, 2025, URL <http://arxiv.org/abs/2507.12233>. [arXiv:2507.12233](https://arxiv.org/abs/2507.12233) [cs].
- [56] Shizheng Wen, Mingyuan Chi, Tianwei Yu, Ben Moseley, Mike Yan Michelis, Pu Ren, Hao Sun, Siddhartha Mishra, Learning, solving and optimizing PDEs with TensorGalerkin: an efficient high-performance Galerkin assembly algorithm, 2026, URL <http://arxiv.org/abs/2602.05052>. [arXiv:2602.05052](https://arxiv.org/abs/2602.05052) [cs].
- [57] South Cass Avenue, PETSc Users Manual.